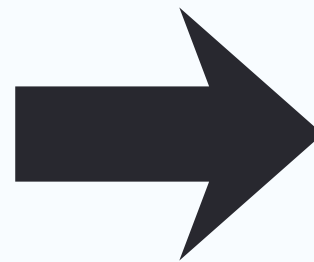
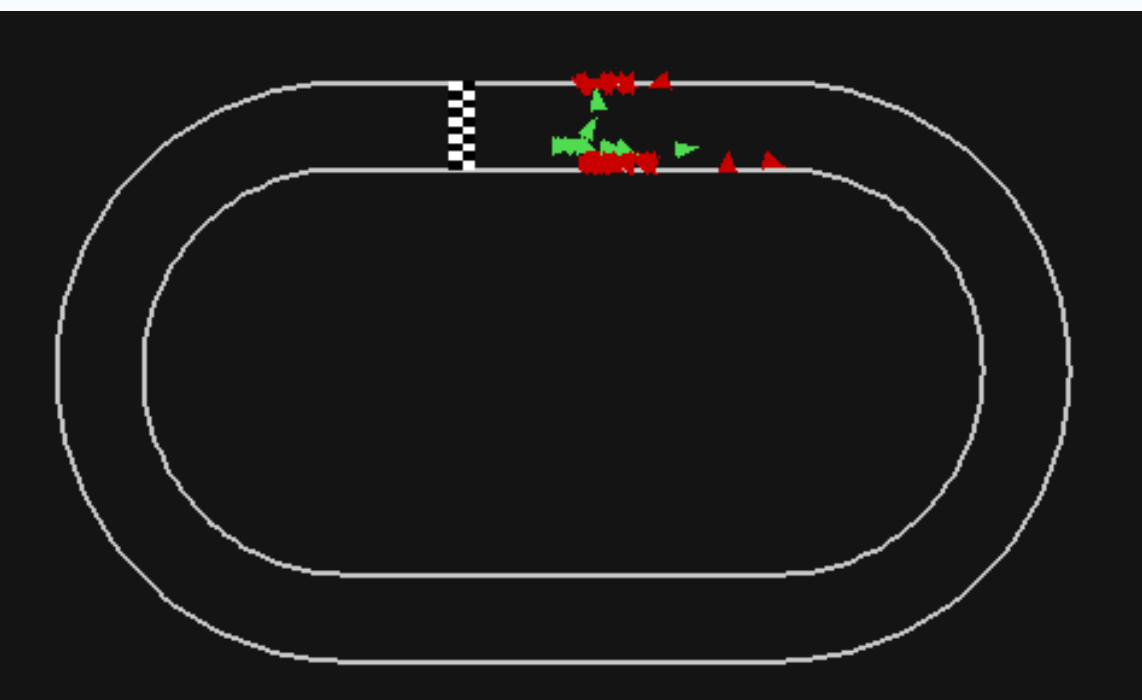


Apprentissage automatique pour la conduite sur circuit



Numéro de candidat : 40107

PROBLÉMATIQUE :

Comment différents algorithmes d'apprentissage permettent-ils à un agent autonome d'apprendre à parcourir un circuit ?

Lequel de ces algorithmes est le meilleur ?

Quelle est la meilleure méthode d'apprentissage ?

Plan

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

5. Amélioration du réalisme

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

5. Amélioration du réalisme

6. Méthodes d'apprentissage

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

5. Amélioration du réalisme

6. Méthodes d'apprentissage

7. Comparaisons

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

5. Amélioration du réalisme

6. Méthodes d'apprentissage

7. Comparaisons

8. Pistes possibles

Plan

1. Généralités :

- Un agent: Kezako?
- Environnement et perception

2. Q-learning

3. Limites & nouvelle représentation :

- Ancienne représentation
- Nouvelle représentation

4. Algorithme génétique et réseaux de neurones :

- Algorithme génétique
- Réseau de neurones
- Reproduction
- Mutation

5. Amélioration du réalisme

6. Méthodes d'apprentissage

7. Comparaisons

8. Pistes possibles

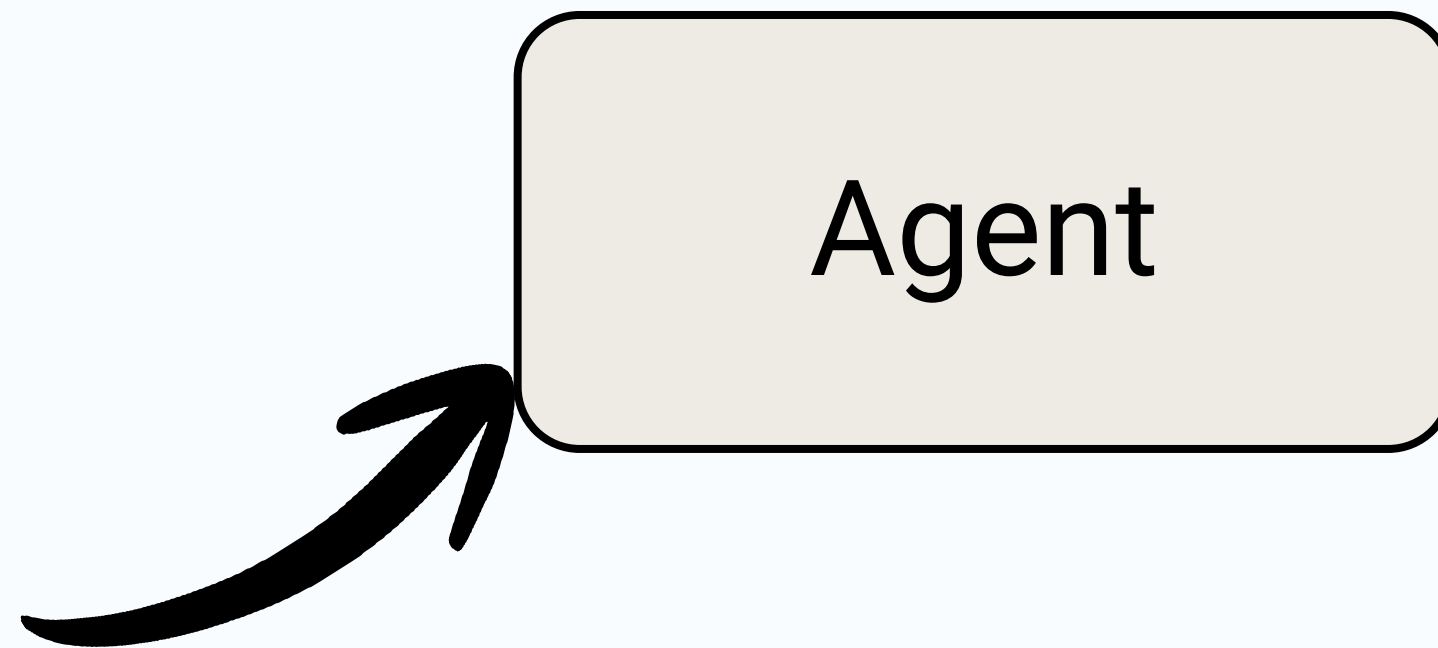
9. Conclusion et expérience finale

Un agent : kézako?



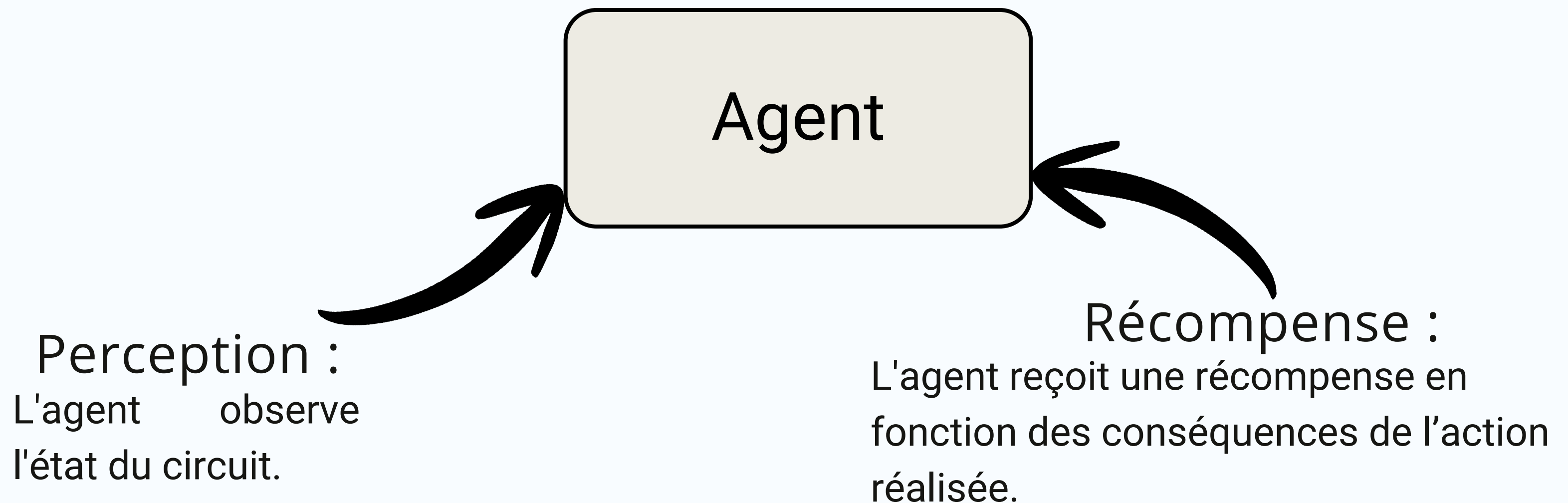
Agent

Un agent : kézako?

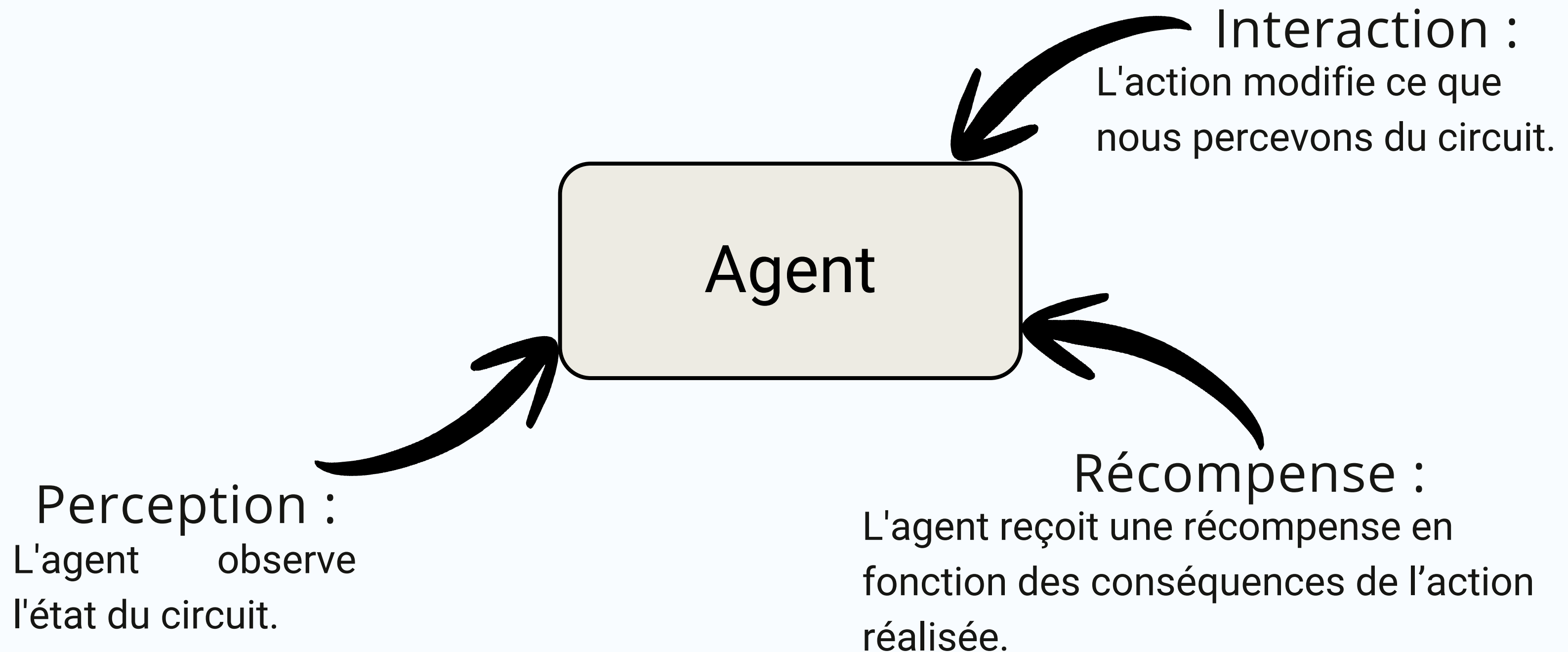


Perception :
L'agent observe
l'état du circuit.

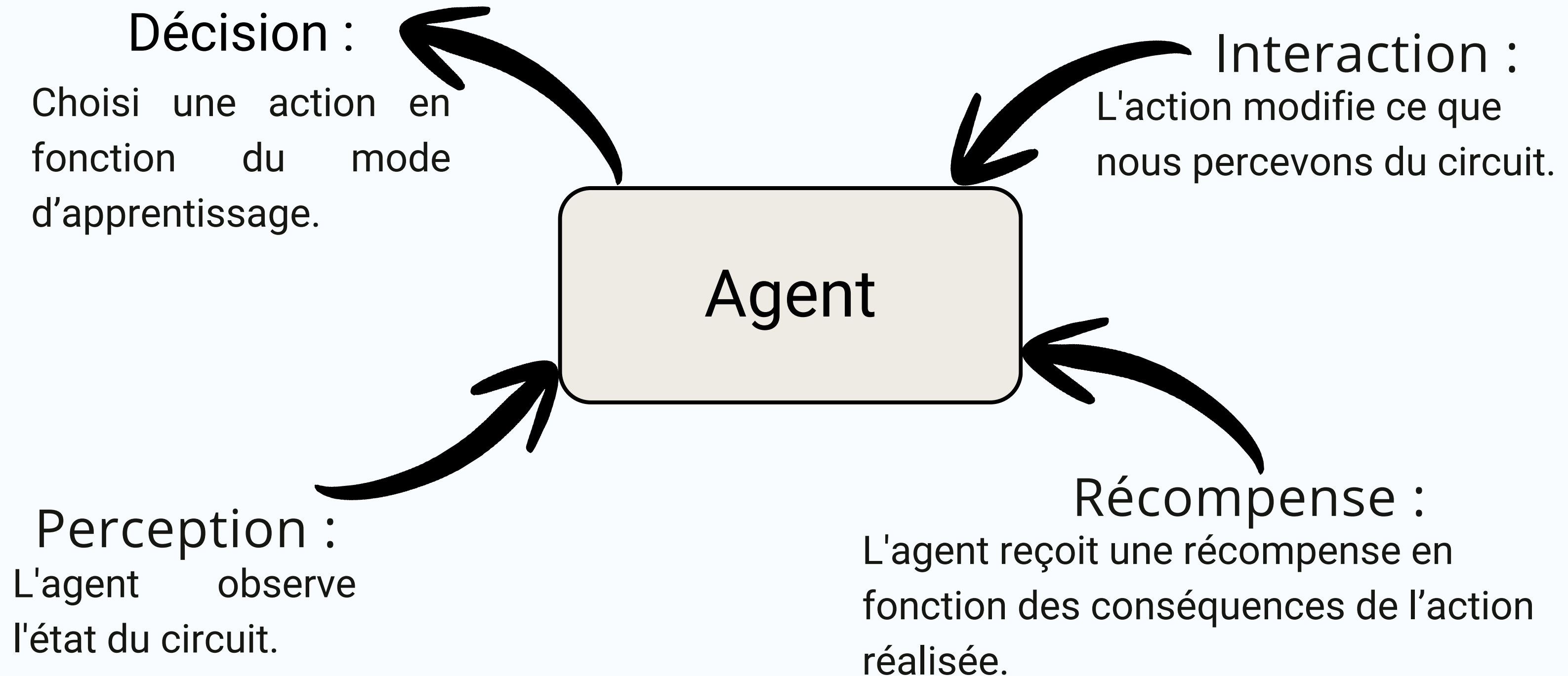
Un agent : kézako?



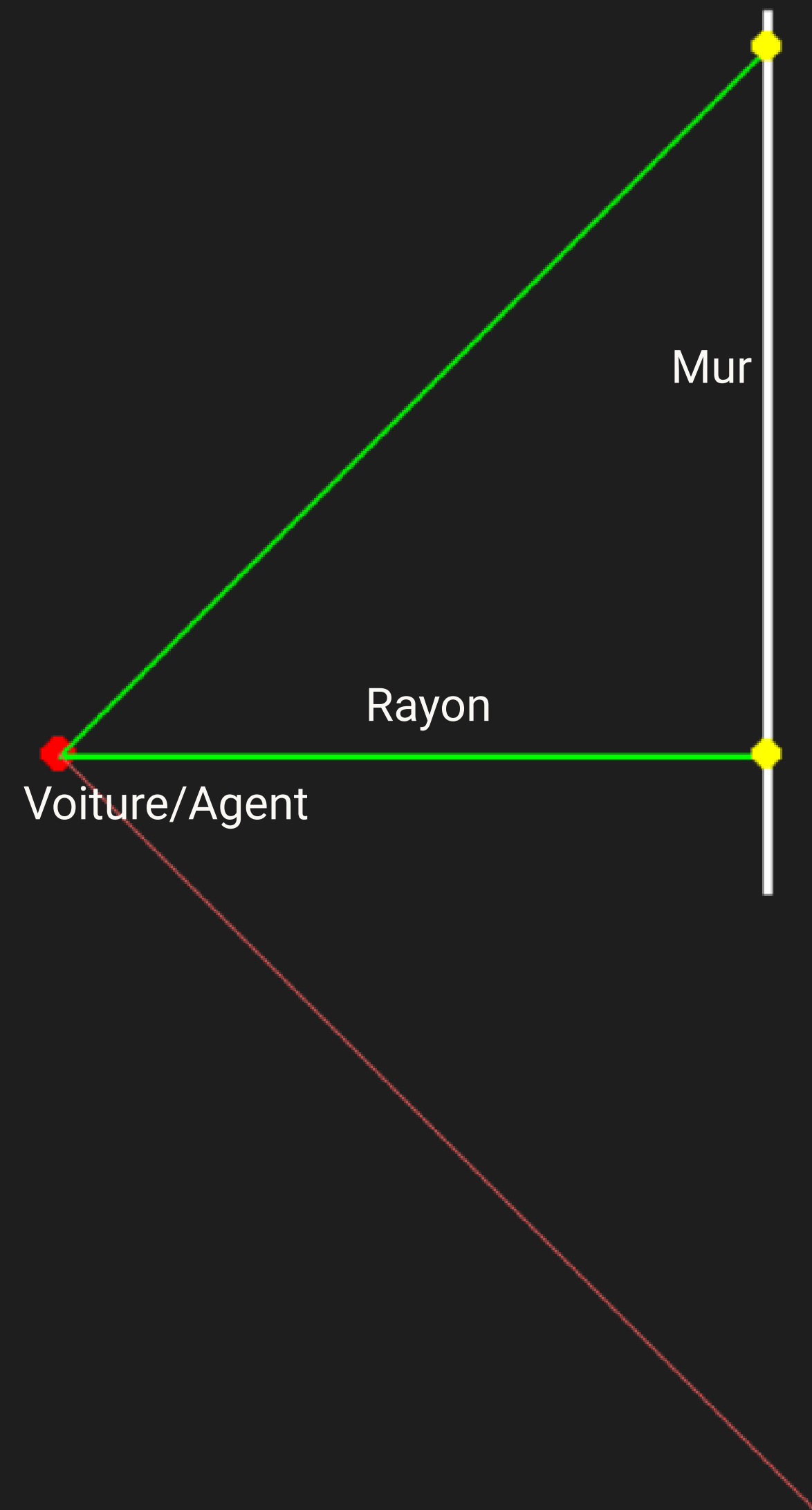
Un agent : kézako?



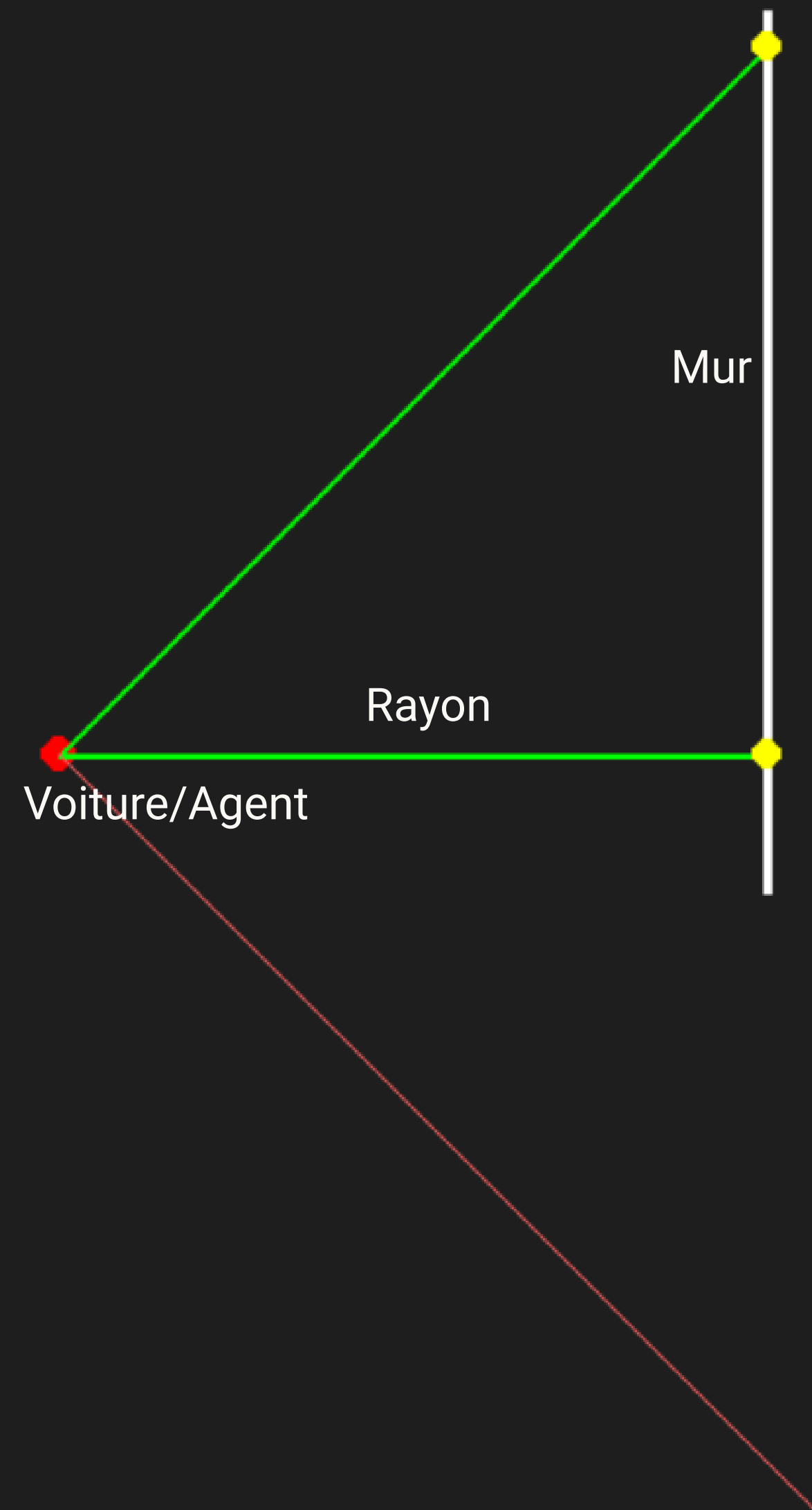
Un agent : k zak ?



Environnement de simulation



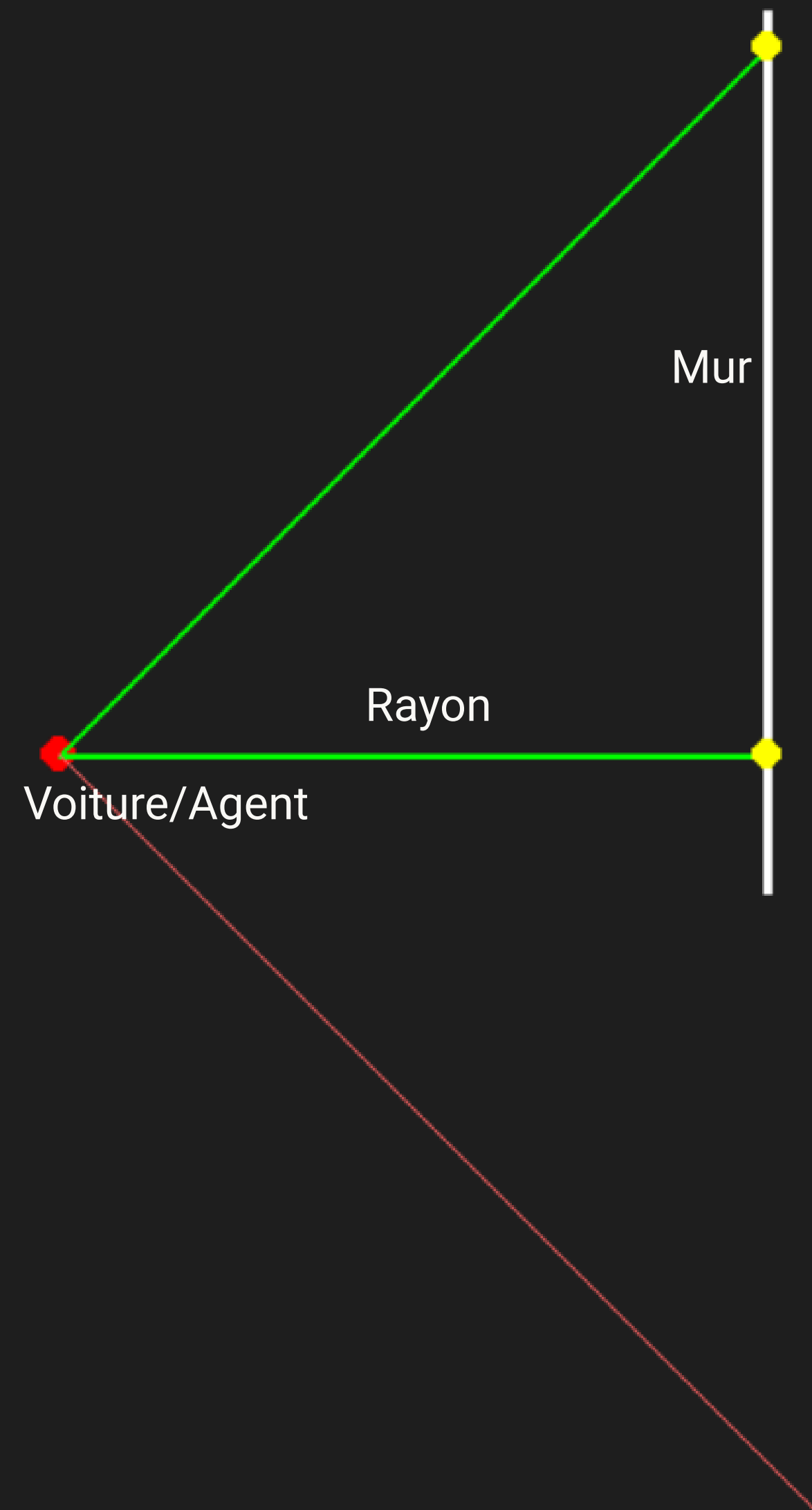
Environnement de simulation



Capteurs :

- Distances aux murs
- Vitesse actuelle
- Angle

Environnement de simulation



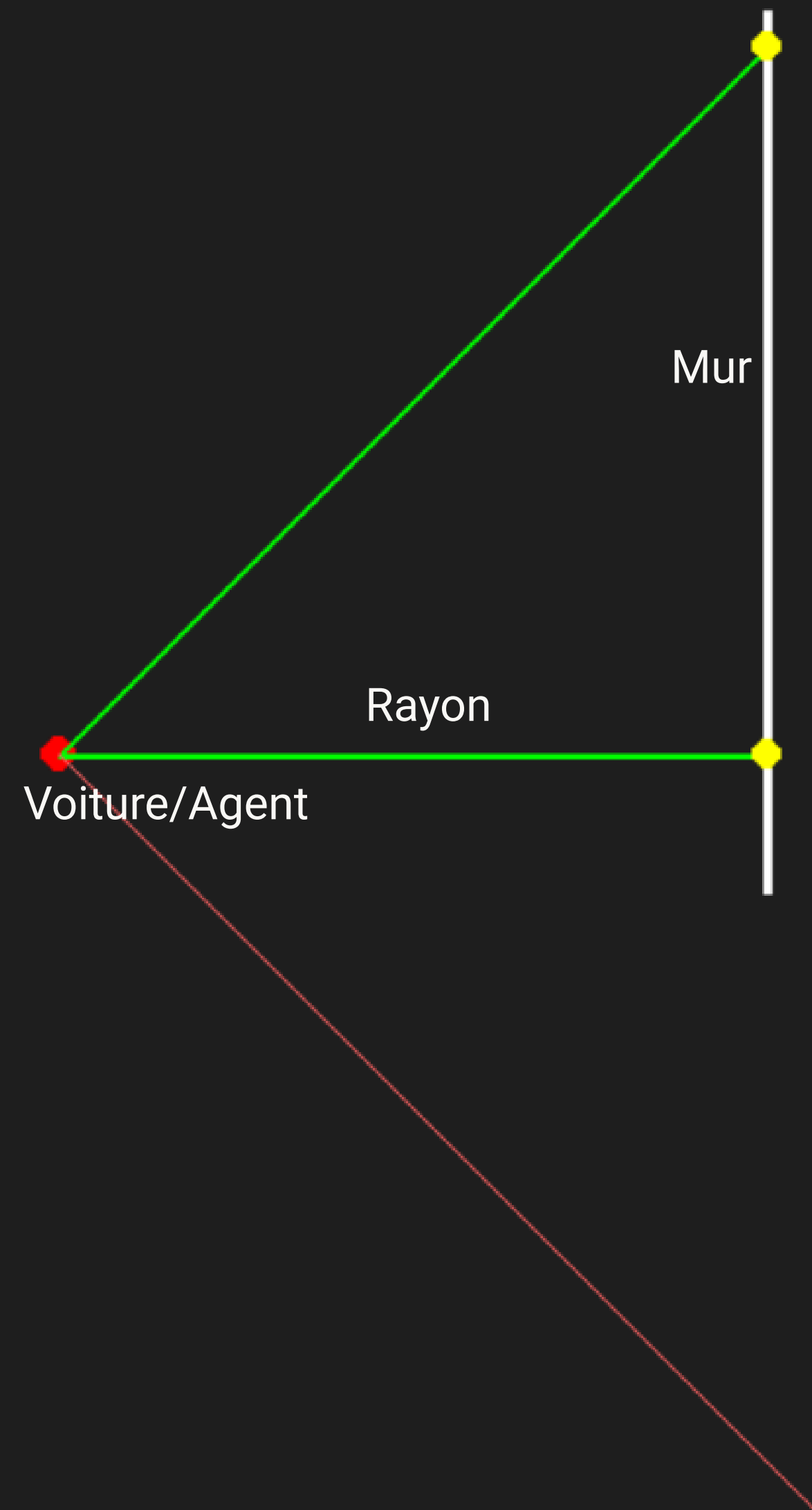
Capteurs :

- Distances aux murs
- Vitesse actuelle
- Angle

Actions :

- Tourner droite/gauche
- Avancer
- Freiner

Environnement de simulation



Capteurs :

- Distances aux murs
- Vitesse actuelle
- Angle

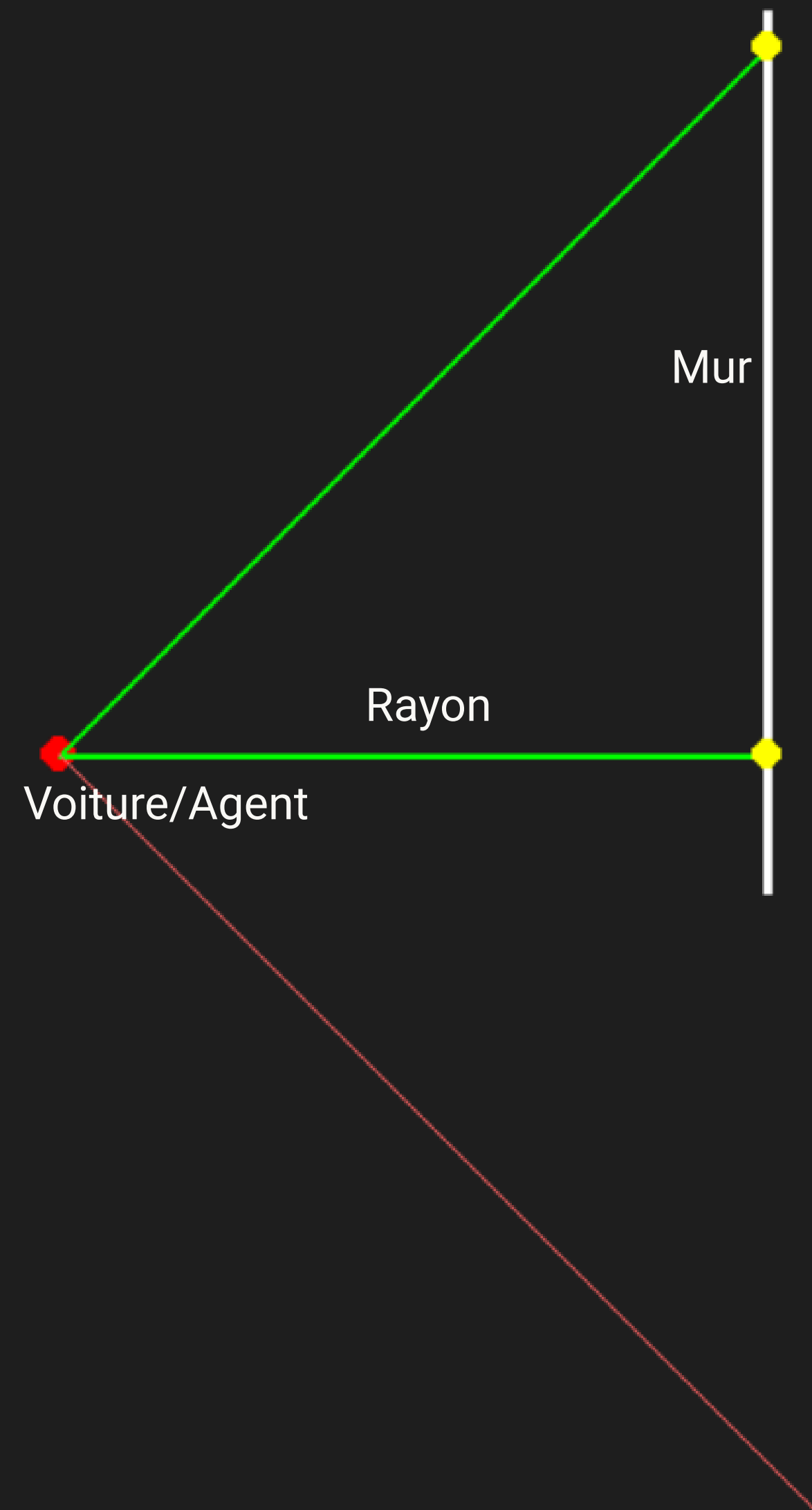
Actions :

- Tourner droite/gauche
- Avancer
- Freiner

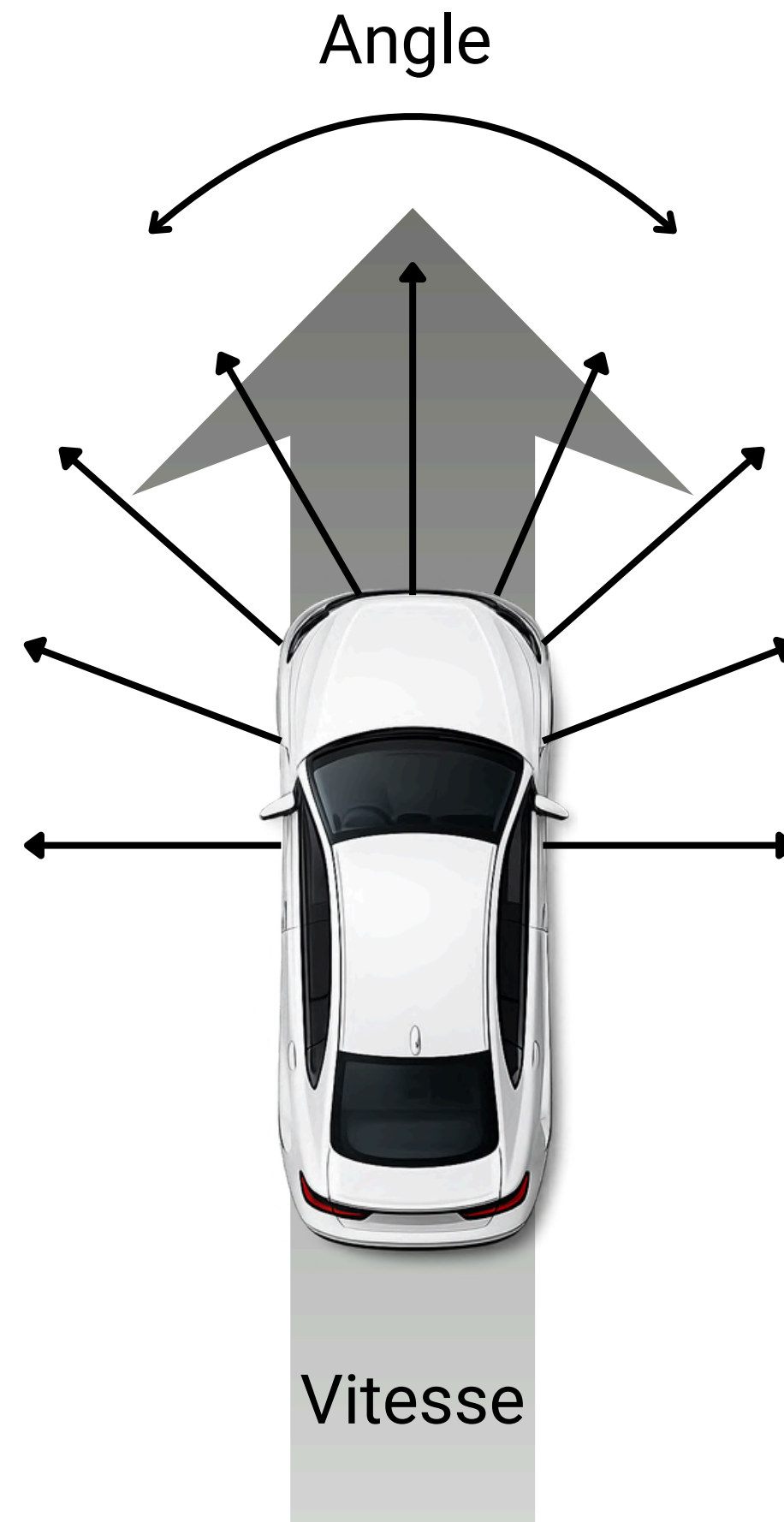
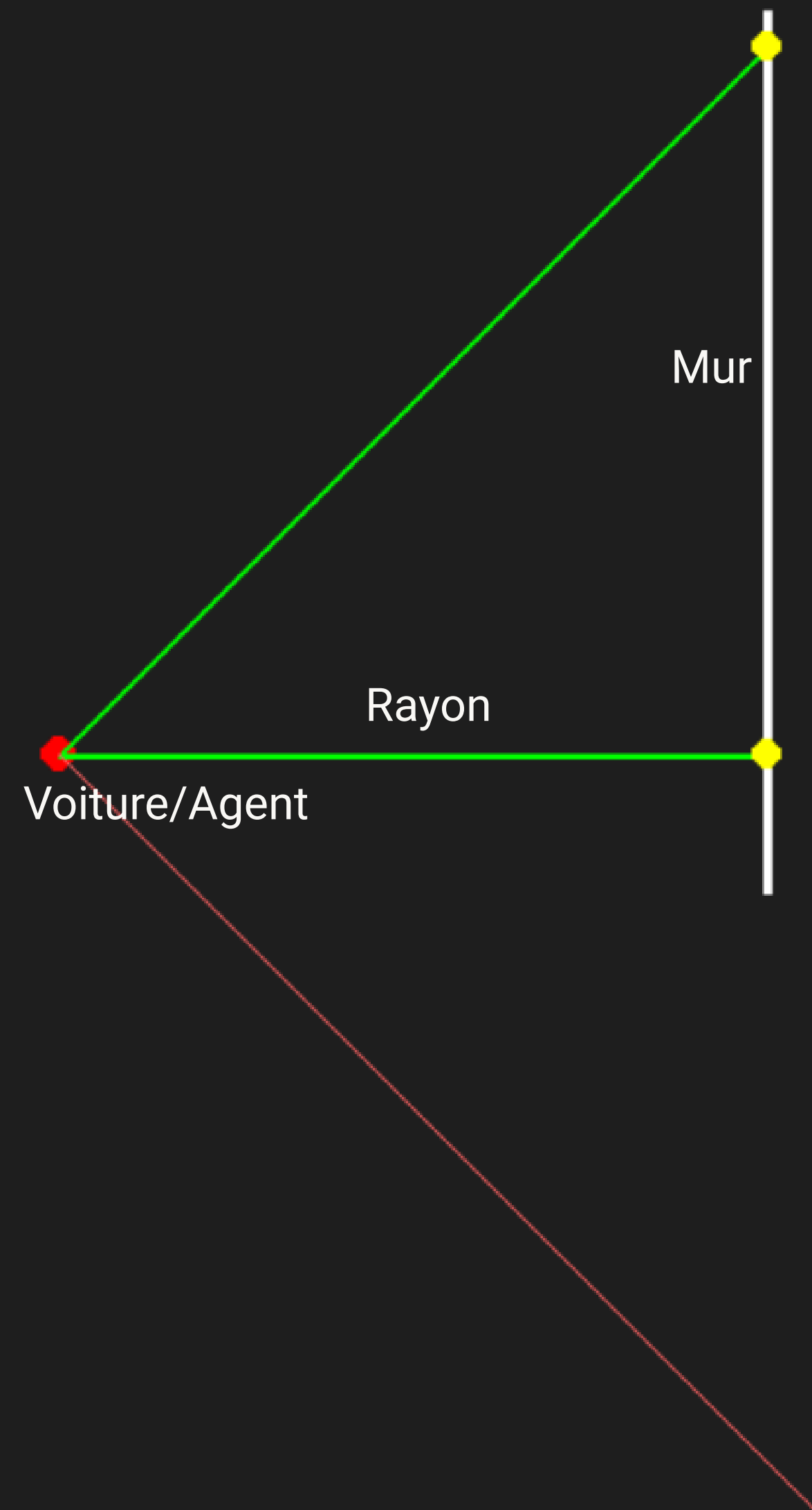
Récompenses :

- + Avancer sur la piste et continuer le circuit
- Se rapprocher d'un mur
- - Collisions

Environnement de simulation



Environnement de simulation



Méthode 1 : Q-learning

Exemple de Q-table :

State, Action, Value :

"[0.3, 0.5, 0.2, 0.2, 0.2, 0.2, 0.4]", 0, 8.3

"[0.3, 0.5, 0.2, 0.2, 0.2, 0.2, 0.4]", 1, 8.1

"[0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.2]", 0, 1.8

"[0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.3]", 0, 1.9

En pratique :

- Une **table** mémorise la valeur de chaque action selon la situation
- L'agent explore puis exploite les meilleures actions
- α : taux d'apprentissage
- γ : facteur de prévision

Méthode 1 : Q-learning

Formule de mise à jour : $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Exemple de Q-table :

State, Action, Value :

"[0.3, 0.5, 0.2, 0.2, 0.2, 0.2, 0.4]", 0, 8.3

"[0.3, 0.5, 0.2, 0.2, 0.2, 0.2, 0.4]", 1, 8.1

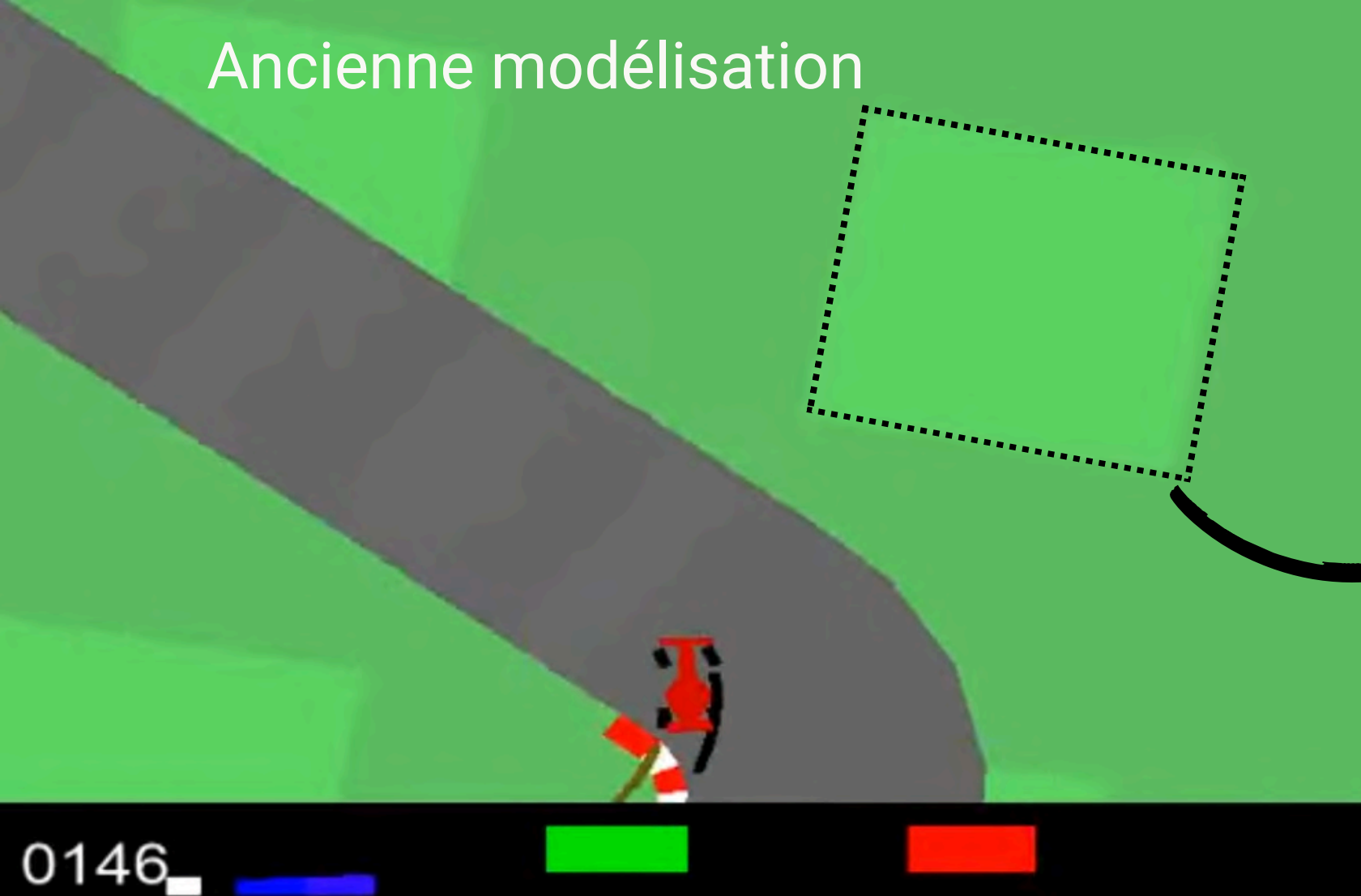
"[0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.2]", 0, 1.8

"[0.1, 0.2, 0.1, 0.1, 0.1, 0.1, 0.3]", 0, 1.9

En pratique :

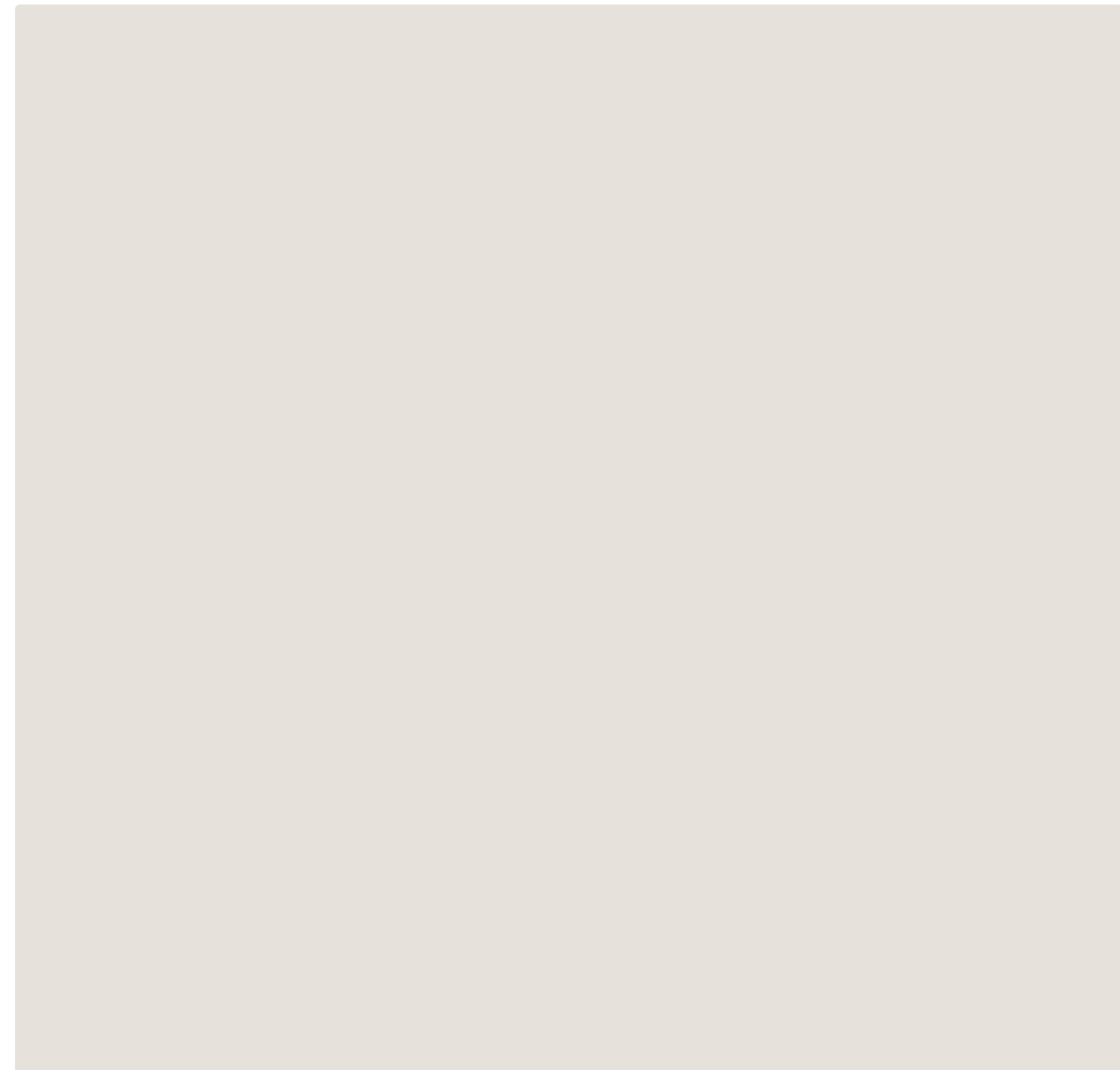
- Une **table** mémorise la valeur de chaque action selon la situation
- L'agent explore puis exploite les meilleures actions
- α : taux d'apprentissage
- γ : facteur de prévision

Ancienne modélisation

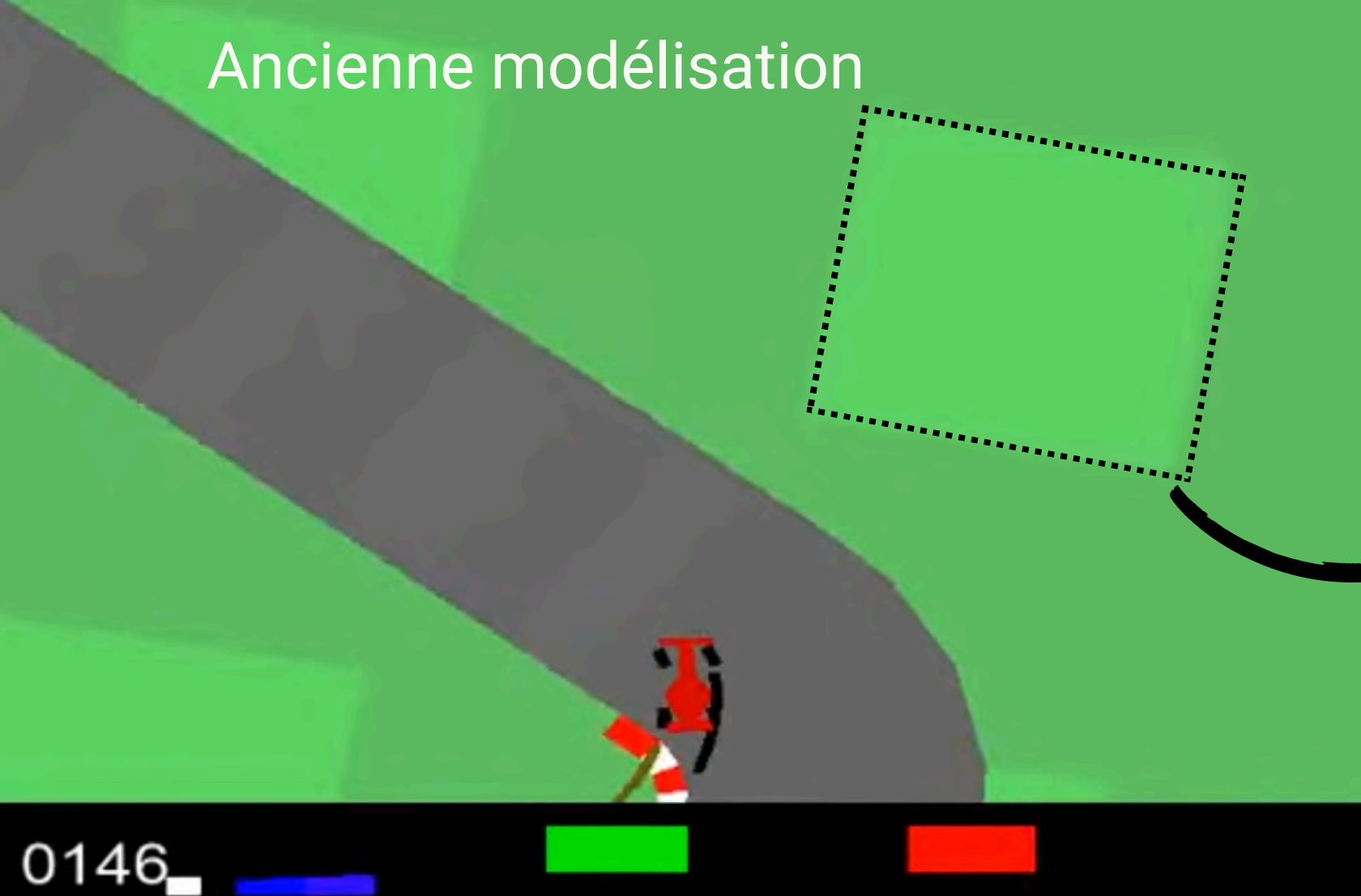


Limites & nouvelle représentation

Pixel : Problème de zoom



Ancienne modélisation

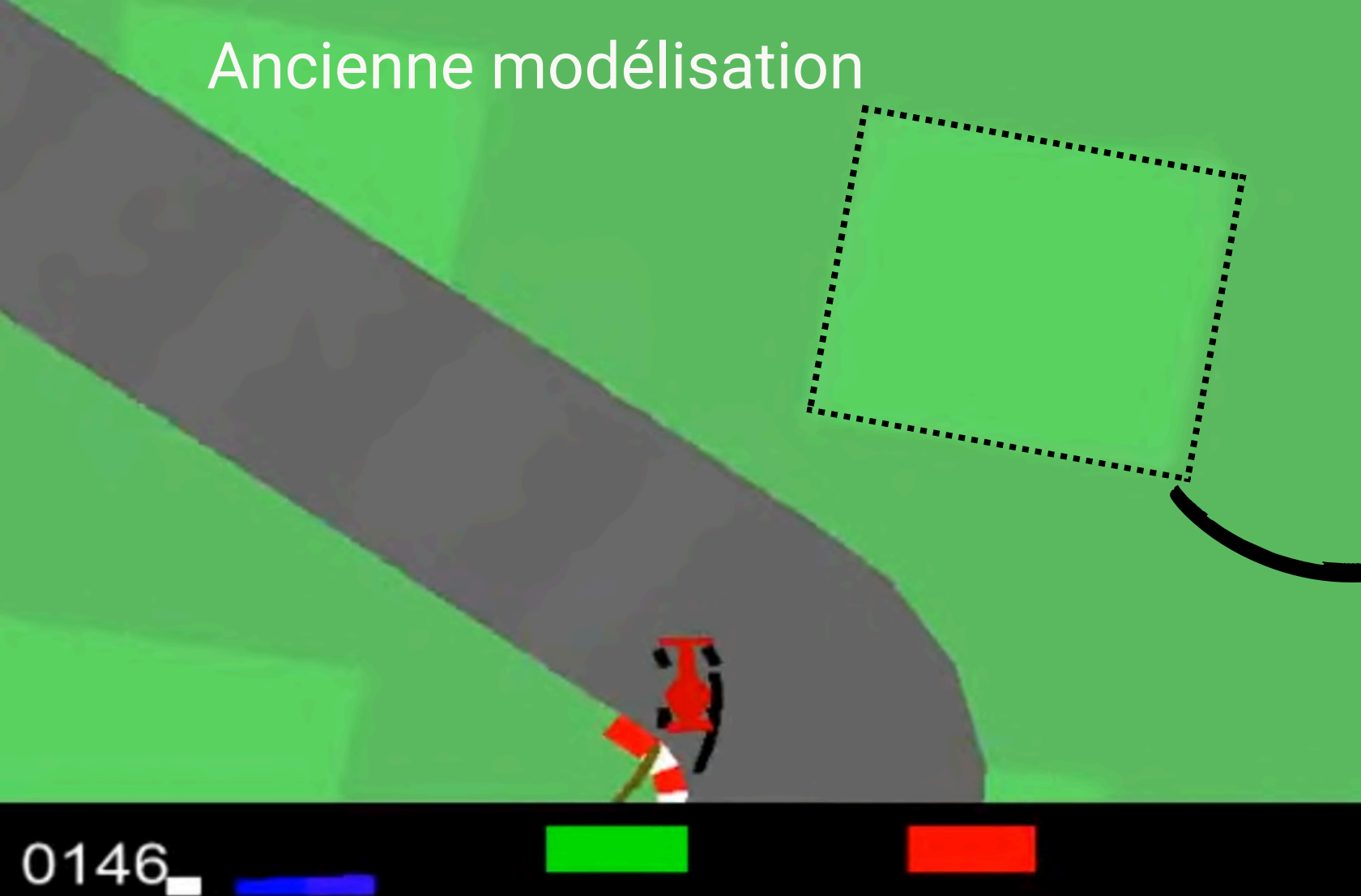


Limites & nouvelle représentation

Pixel : Problème de zoom

- Généralisation impossible
- Temps d'entraînement long

Ancienne modélisation

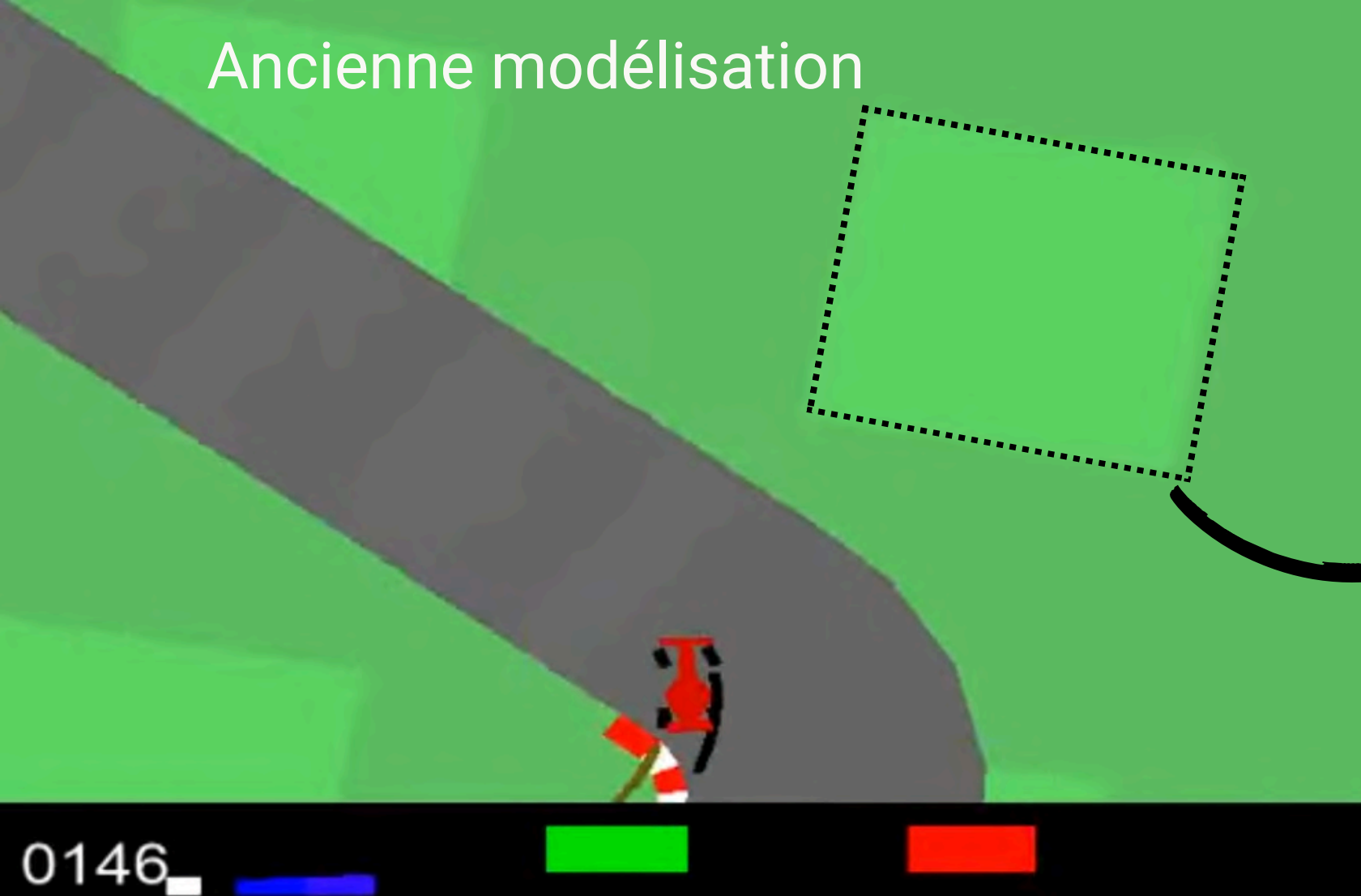


Limites & nouvelle représentation

Pixel : Problème de zoom

- Généralisation impossible
- Temps d'entraînement long
- Peu précis
- Non détection de sortie de piste

Ancienne modélisation

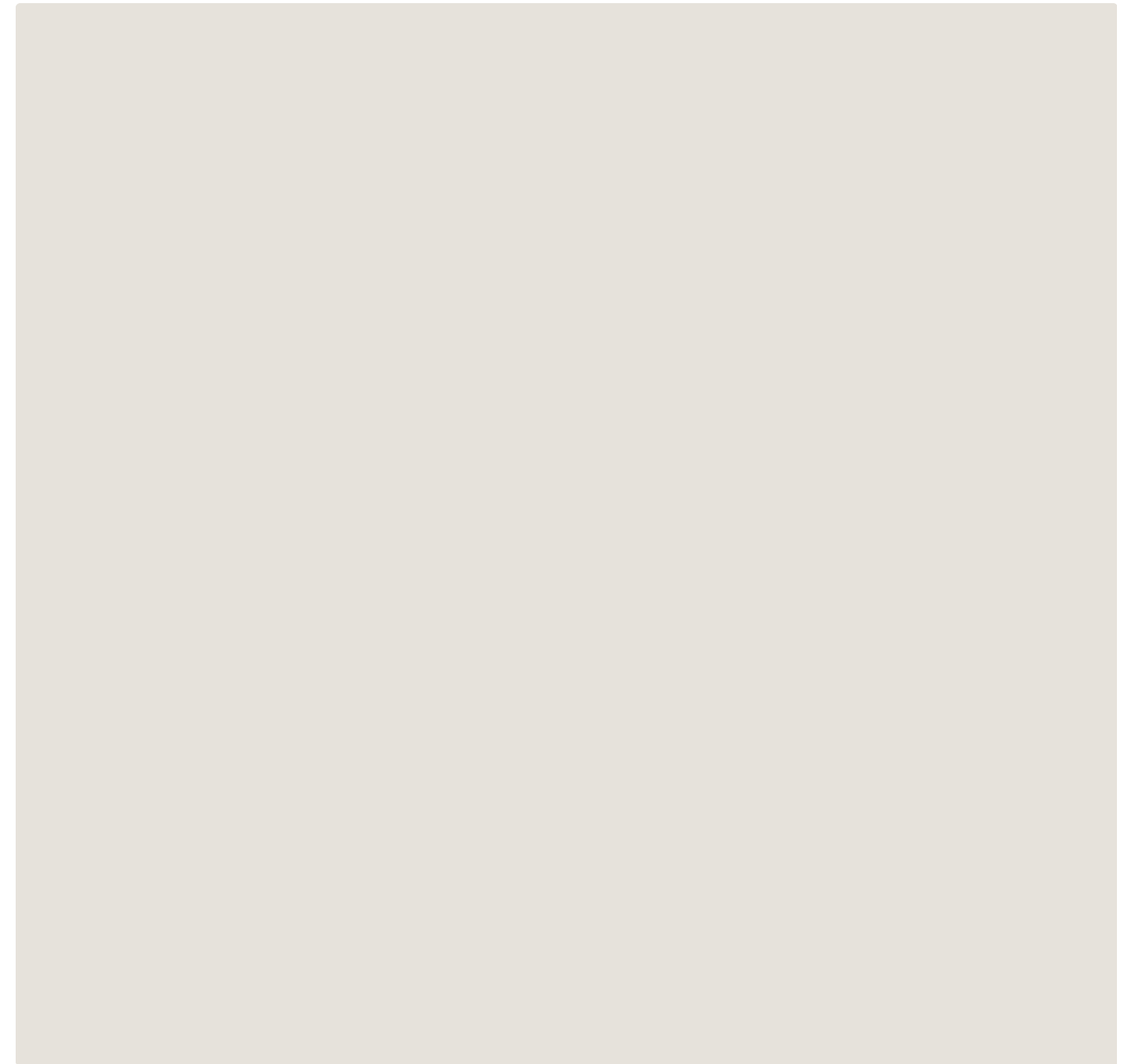


Limites & nouvelle représentation

Pixel : Problème de zoom

- Généralisation impossible
- Temps d'entraînement long
- Peu précis
- Non détection de sortie de piste
- Physique non appropriée
- Impossibilité de paramétrer les récompenses

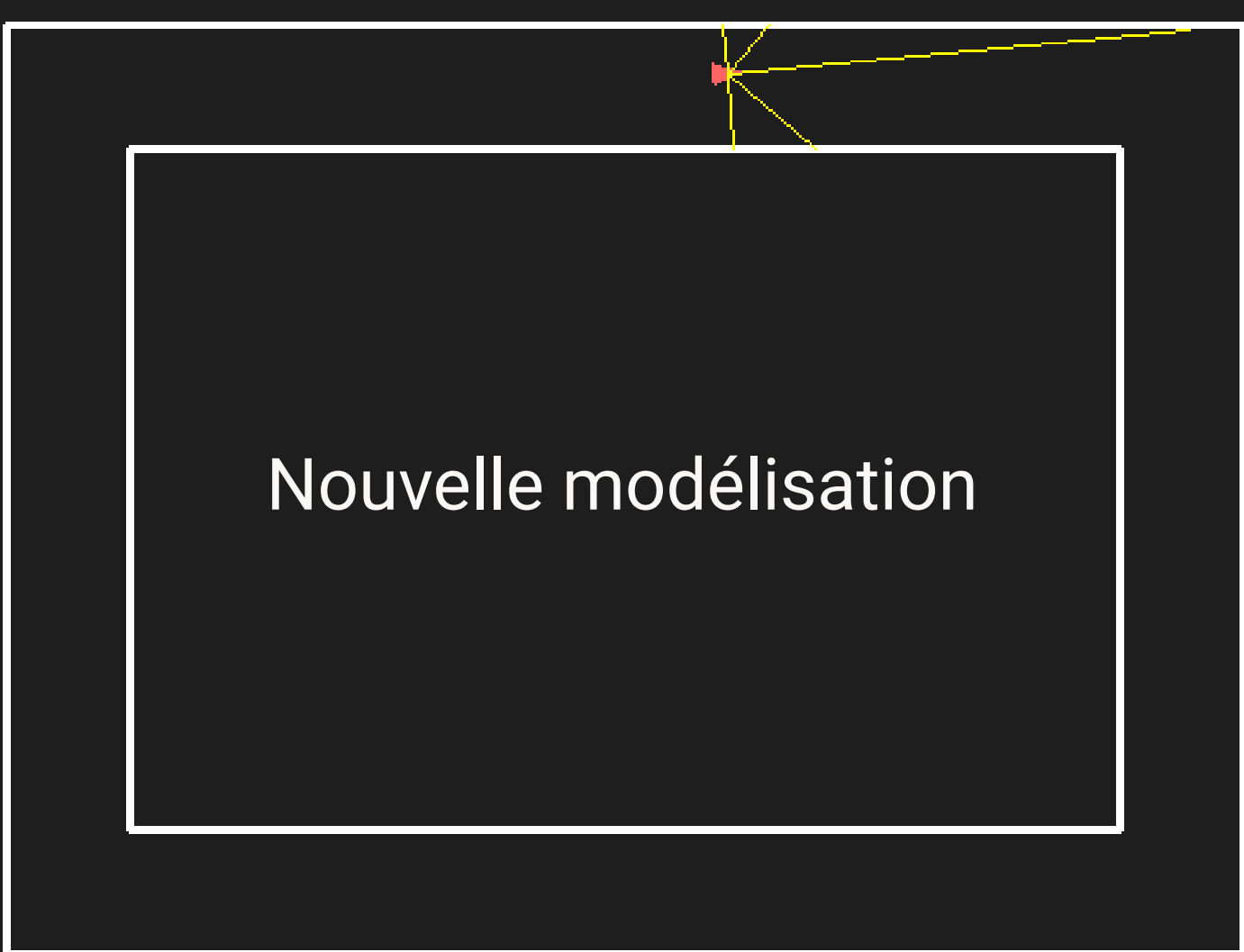
Limites & nouvelle représentation



Nouvelle modélisation

Limites & nouvelle représentation

- Création de nos propres circuits

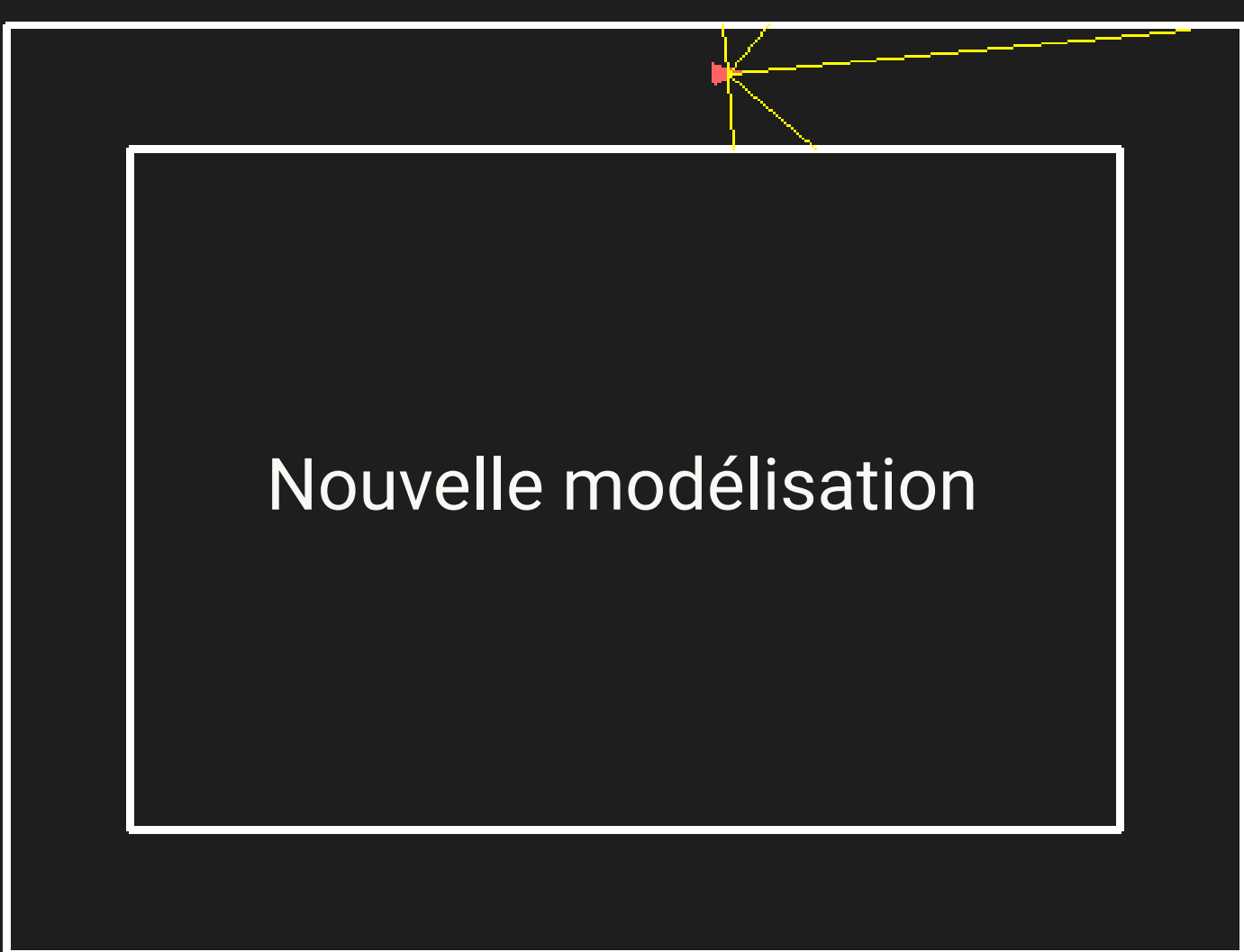


Nouvelle modélisation

The diagram shows a dark gray rectangular area with a white border. Inside this area is a smaller dark gray rectangle with a white border. At the top center of the inner rectangle, there is a small red square node. From this node, several yellow lines radiate outwards, connecting to the top and right edges of the inner rectangle. The text 'Nouvelle modélisation' is written in white in the center of the inner rectangle.

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances



Nouvelle modélisation

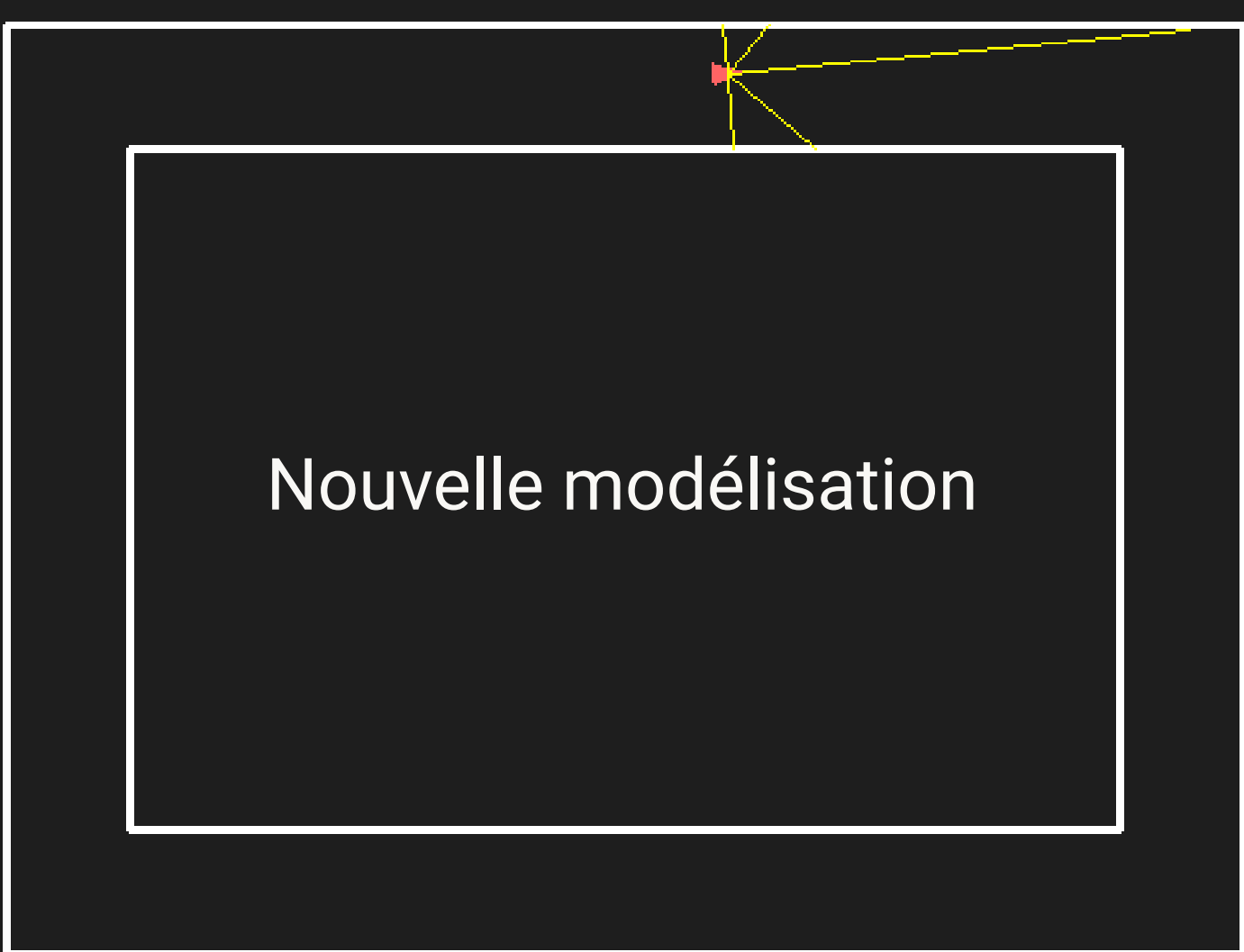
The diagram shows a dark gray rectangular area with a white border. Inside this area is a smaller dark gray rectangle with a white border. At the top center of the inner rectangle, there is a small red square node. From this node, several yellow lines radiate outwards, connecting to the top and right edges of the inner rectangle. The text 'Nouvelle modélisation' is written in white in the center of the inner rectangle.

- A et B : points aux extrémités du murs
- u dans [0;1]
- O l'origine
- t = distance cherchée
- d = direction du rayon
- v = B-A = le vecteur de A vers B
- On cherche t = u qui désigne le même point

$$O + td = A + uv$$

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances
- Intersection des rayons avec les murs



The diagram shows a dark gray rectangular room with a white border. Inside, there is a smaller white rectangle. A yellow ray originates from a red point on the top wall of the inner rectangle and extends towards the top-right corner. A small yellow crosshair marks the intersection of the ray with the top wall.

Nouvelle modélisation

- A et B : points aux extrémités du murs
- u dans [0;1]
- O l'origine
- t = distance cherchée
- d = direction du rayon
- v = B-A = le vecteur de A vers B
- On cherche t = u qui désigne le même point

$$O + td = A + uv$$

↔ $t = \frac{(A - O) \cdot v}{d \cdot v}$ et $u = \frac{(A - O) \cdot d}{d \cdot v}$

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances
- Intersection des rayons avec les murs

Nouvelle modélisation

- A et B : points aux extrémités du murs
- u dans [0;1]
- O l'origine
- t = distance cherchée
- d = direction du rayon
- v = B-A = le vecteur de A vers B
- On cherche t = u qui désigne le même point

$$O + td = A + uv$$

↔ $t = \frac{(A - O) \cdot v}{d \cdot v}$ et $u = \frac{(A - O) \cdot d}{d \cdot v}$

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances
- Intersection des rayons avec les murs
- Récompenses paramétrées comme souhaitées

Nouvelle modélisation

- A et B : points aux extrémités du murs
- u dans [0;1]
- O l'origine
- t = distance cherchée
- d = direction du rayon
- v = B-A = le vecteur de A vers B
- On cherche t = u qui désigne le même point

$$O + td = A + uv$$

↔ $t = \frac{(A - O) \cdot v}{d \cdot v}$ et $u = \frac{(A - O) \cdot d}{d \cdot v}$

Nouvelle modélisation

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances
- Intersection des rayons avec les murs
- Récompenses paramétrées comme souhaitées
- Physique implémentable et modifiable

- A et B : points aux extrémités du murs
- u dans [0;1]
- O l'origine
- t = distance cherchée
- d = direction du rayon
- v = B-A = le vecteur de A vers B
- On cherche t = u qui désigne le même point

$$O + td = A + uv$$

↔ $t = \frac{(A - O) \cdot v}{d \cdot v}$ et $u = \frac{(A - O) \cdot d}{d \cdot v}$

Nouvelle modélisation

Limites & nouvelle représentation

- Création de nos propres circuits
- Normalisation des distances
- Intersection des rayons avec les murs
- Récompenses paramétrées comme souhaitées
- Physique implémentable et modifiable
- Précision accrue

Méthode 2 : Algorithme génétique - Darwin





Méthode 2 : Algorithme génétique - Darwin



Population initiale
Comportements aléatoires





Méthode 2 :

Algorithme génétique - Darwin



Population initiale
Comportements aléatoires



Sélection
Meilleurs agents conservés





Méthode 2 :

Algorithme génétique - Darwin



Population initiale
Comportements aléatoires



Sélection
Meilleurs agents conservés



Mutation
Nouvelles stratégies explorées





Méthode 2 :

Algorithme génétique - Darwin



Population initiale
Comportements aléatoires



Sélection
Meilleurs agents conservés



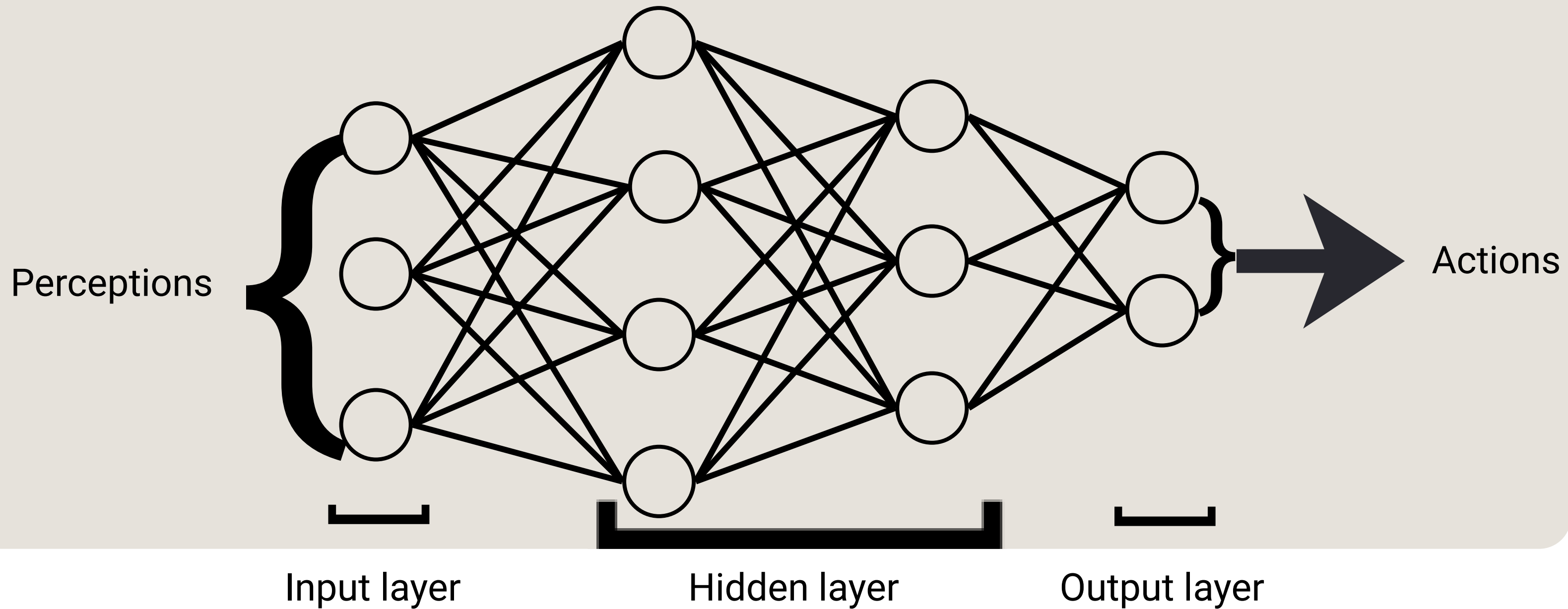
Mutation
Nouvelles stratégies explorées



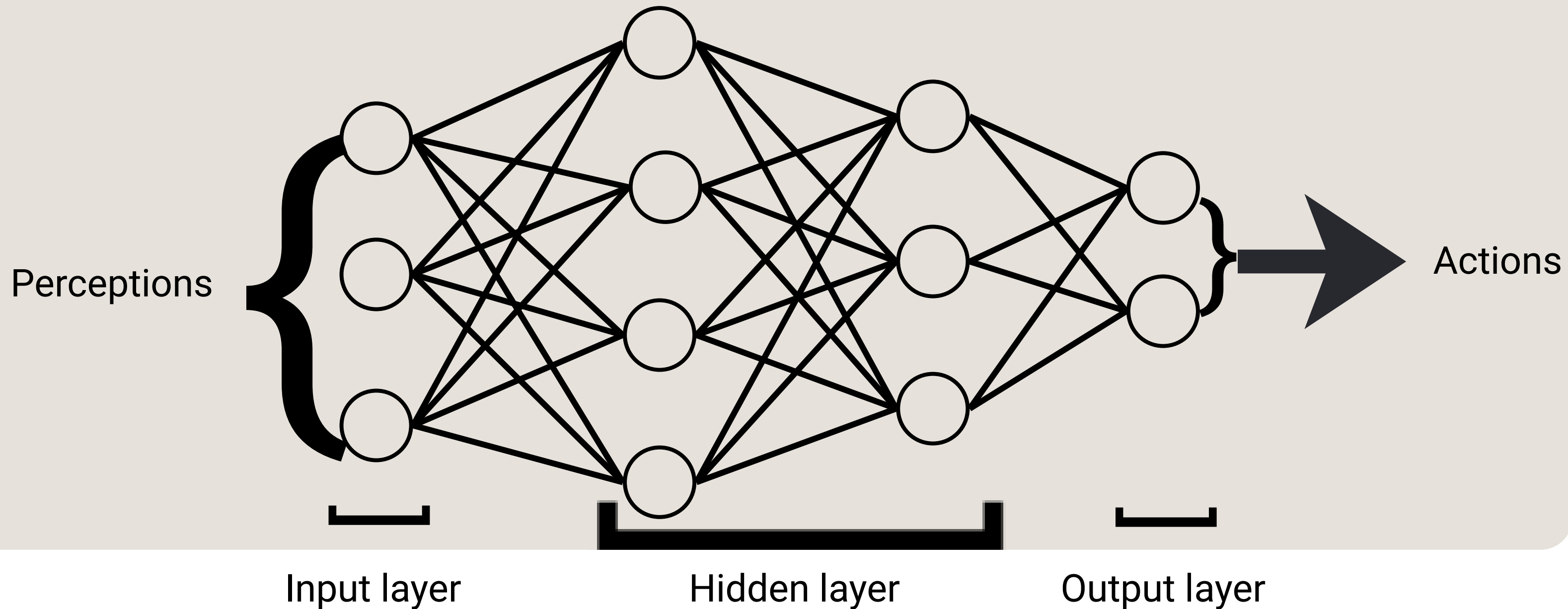
Génération
Amélioration progressive



Réseau de neurones



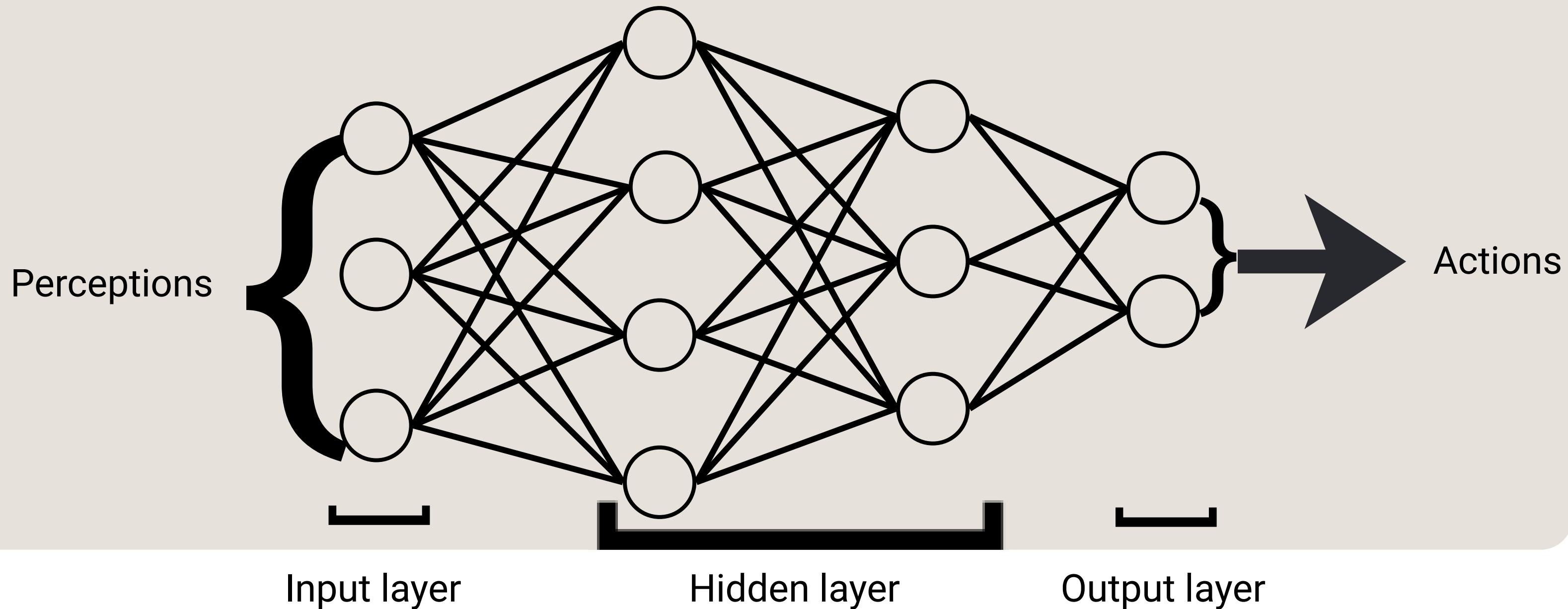
Réseau de neurones



Fonctionnement d'un neurone

Neurone = fonction de calcul qui prend des entrées et produit une décision intermédiaire

Réseau de neurones

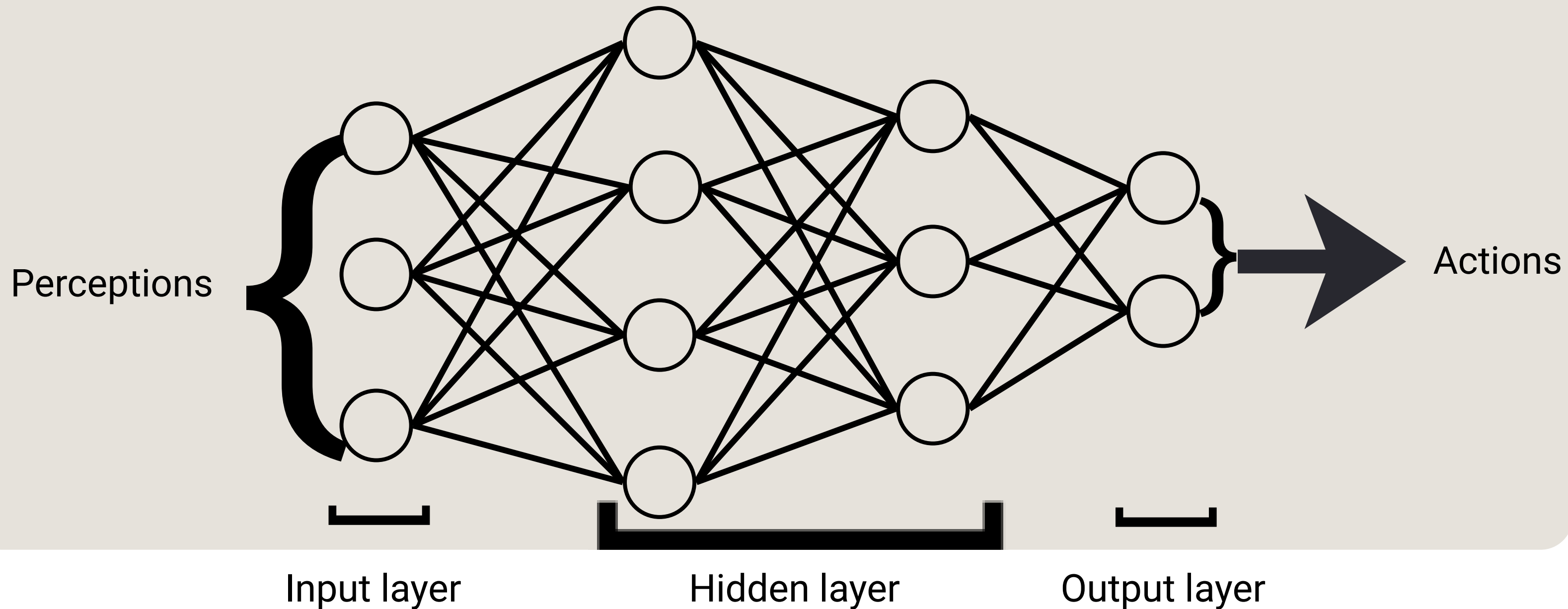


Fonctionnement d'un neurone

Neurone = fonction de calcul qui prend des entrées et produit une décision intermédiaire

Poids = force d'une connection (si un neurone "aime" sa connection avec un autre neurone)

Réseau de neurones



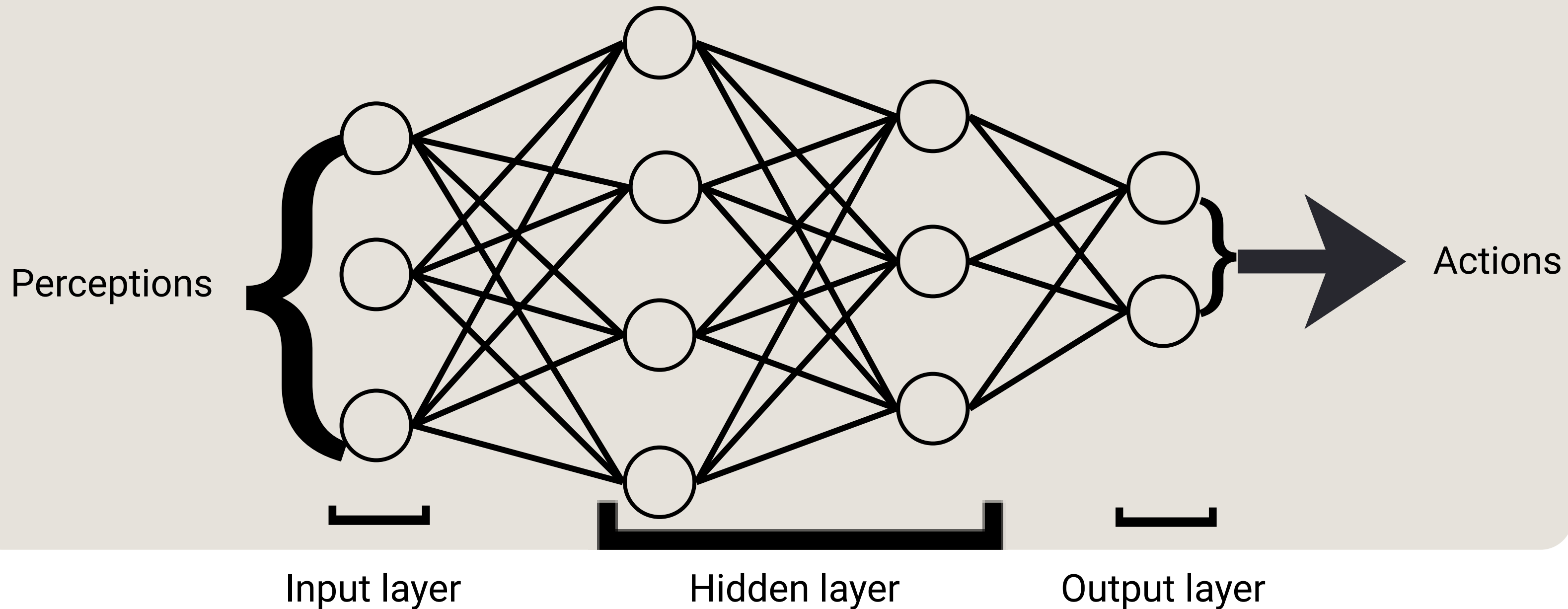
Fonctionnement d'un neurone

Neurone = fonction de calcul qui prend des entrées et produit une décision intermédiaire

Poids = force d'une connection (si un neurone "aime" sa connection avec un autre neurone)

Biais = Seuil d'activation du neurone: on renvoie 0 ou alors ce qui est trouvé

Réseau de neurones



Fonctionnement d'un neurone

Neurone = fonction de calcul qui prend des entrées et produit une décision intermédiaire

Poids = force d'une connection (si un neurone "aime" sa connexion avec un autre neurone)

$$sortie = \left(\sum_{i=entree} information_i * poids_i \right) + biais$$

Biais = Seuil d'activation du neurone: on renvoie 0 ou alors ce qui est trouvé

Réseau de neurones - crossover (reproduction)

Mélange des poids des connexions :

Réseau de neurones - crossover (reproduction)

Mélange des poids des connexions :

Parents :

Parent 1 : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

Parent 2 : [0.7, 0.4, -0.3, 0.6, 0.2, -0.1, ...]

Réseau de neurones - crossover (reproduction)

Mélange des poids des connexions :

Parents :

Parent 1 : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

Parent 2 : [0.7, 0.4, -0.3, 0.6, 0.2, -0.1, ...]

Masque aléatoire :

Masque : [1, 2, 1, 2, 1, 2, ...]

Réseau de neurones - crossover (reproduction)

Mélange des poids des connexions :

Parents :

Parent 1 : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

Parent 2 : [0.7, 0.4, -0.3, 0.6, 0.2, -0.1, ...]

Masque aléatoire :

Masque : [1, 2, 1, 2, 1, 2, ...]

Poids de l'enfant :

Enfant : [0.3, 0.4, 0.8, 0.6, -0.5, -0.1, ...]

Réseau de neurones - mutation

Mutation rate = fréquence de mutation

Mutation strenght = force/capacité de mutation

Réseau de neurones - mutation

Mutation rate = fréquence de mutation

Mutation strenght = force/capacité de mutation

Poids d'un agent : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

Réseau de neurones - mutation

Mutation rate = fréquence de mutation

Mutation strenght = force/capacité de mutation

Poids d'un agent : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

+

Mutation rate = 30%

Mutation strenght = 2 $\rightarrow$ [-2; 2]

Réseau de neurones - mutation

Mutation rate = fréquence de mutation

Mutation strenght = force/capacité de mutation

Poids d'un agent : [0.3, -1.2, 0.8, 0.1, -0.5, 0.9, ...]

+

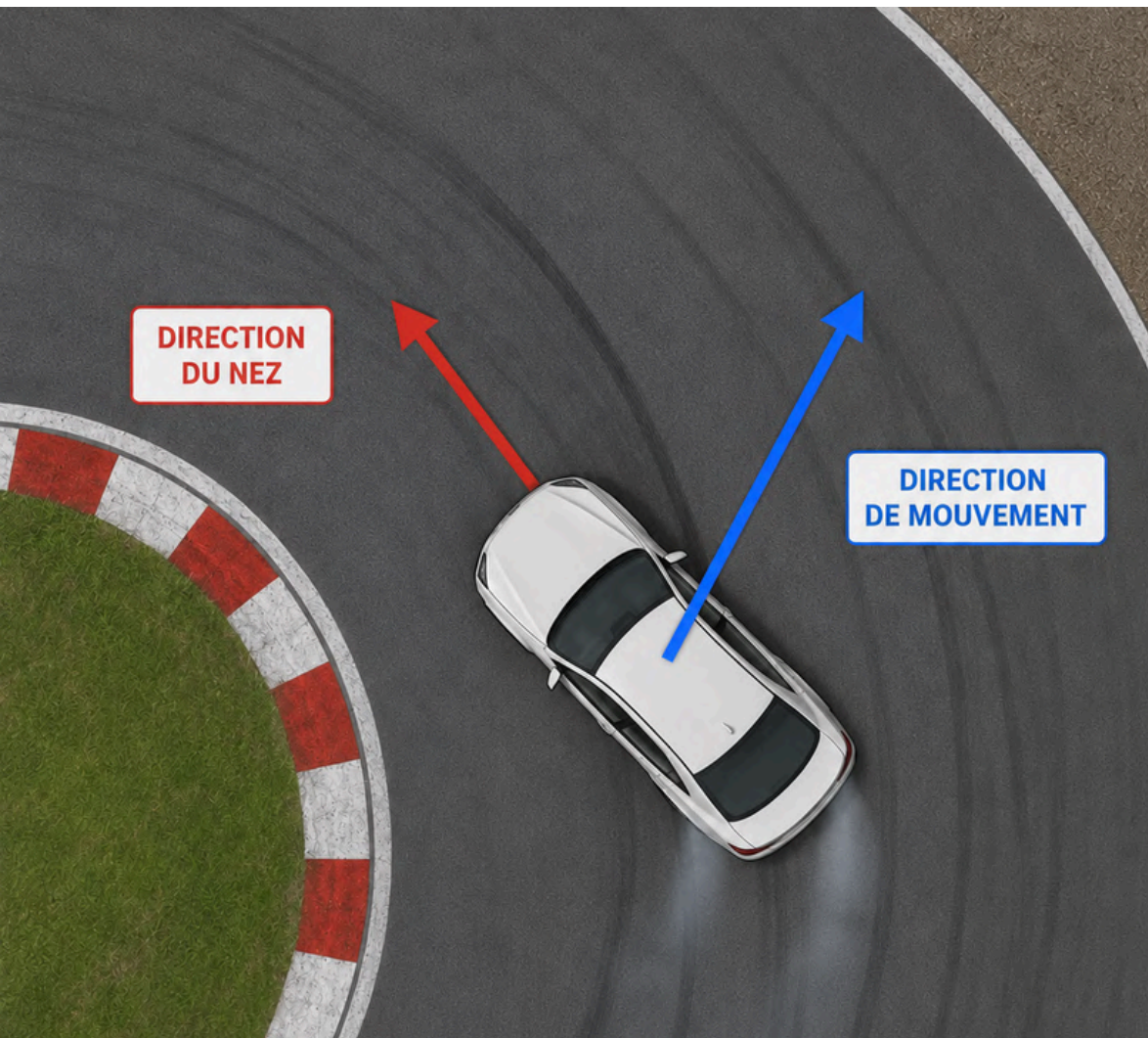
Mutation rate = 30%

Mutation strenght = 2 $\rightarrow$ [-2; 2]



Poids de l'agent muté : [0.3, -3.0, 1.7, 0.1, 0, 0.9, ...]

Souci de réalisme : ajout de la physique



- Création de différents pneus et leur usure
- Création d'un réservoir d'essence limité
- Ajout des forces de frottements (possibilité de drift)
- Ajout d'actions (2 types de pits stop)
- Curriculum learnings

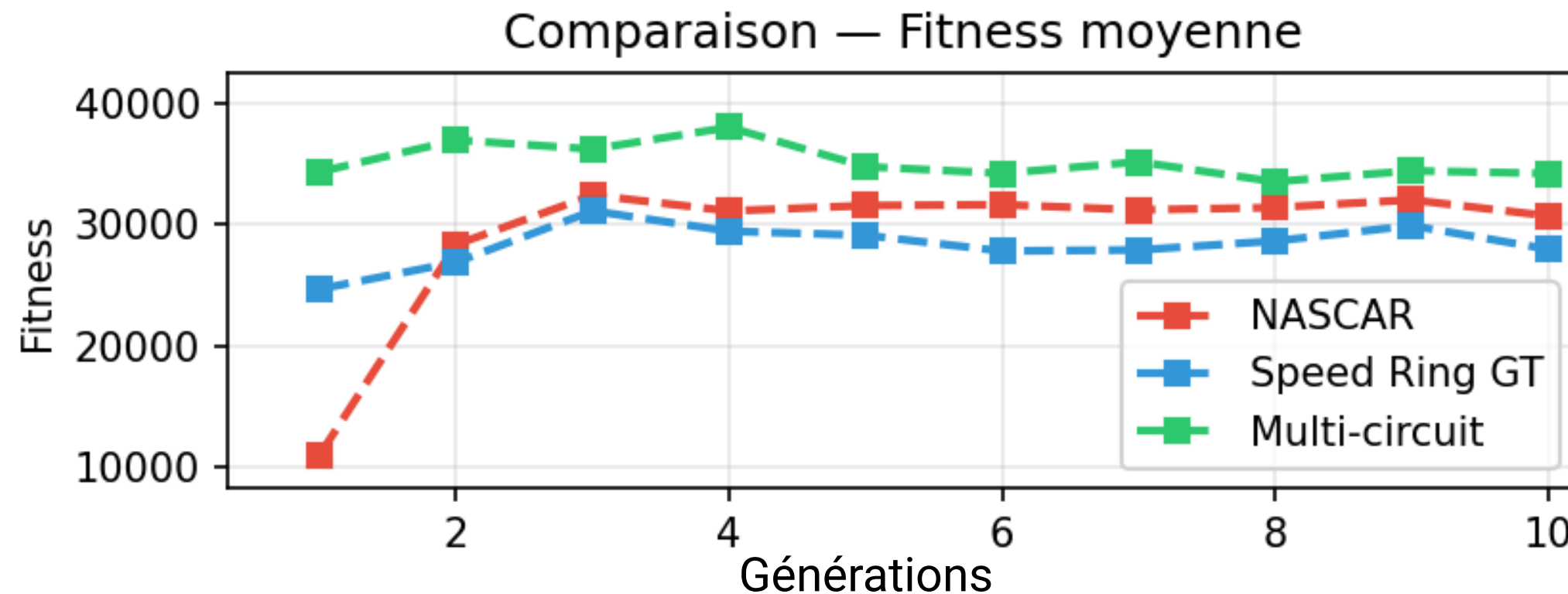


Réalisme amélioré



Apprentissage : Mono VS Multi-circuits

Essaies de 3 populations sur un nouveau circuit

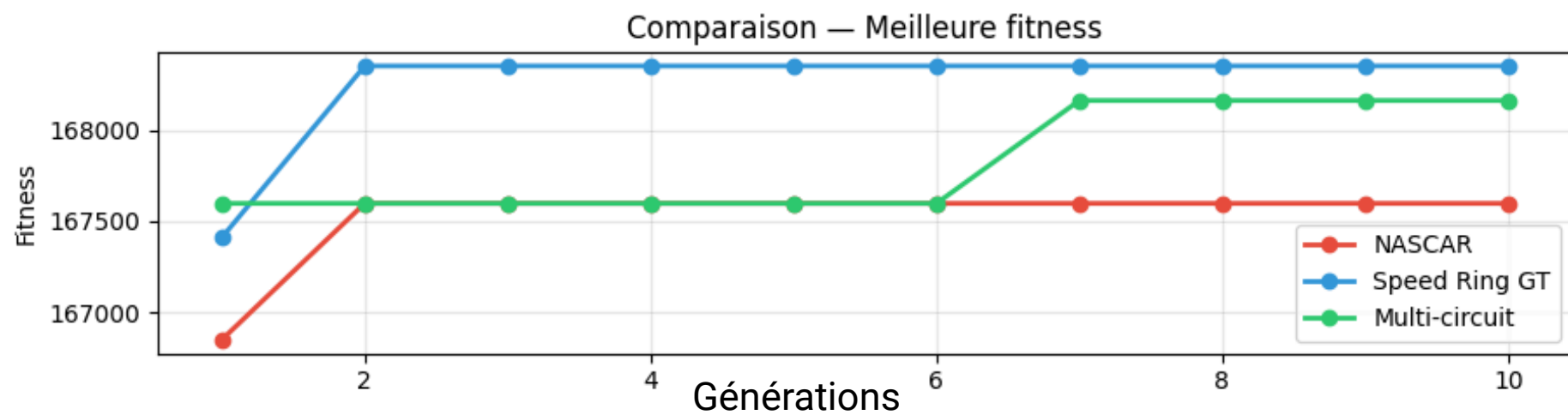
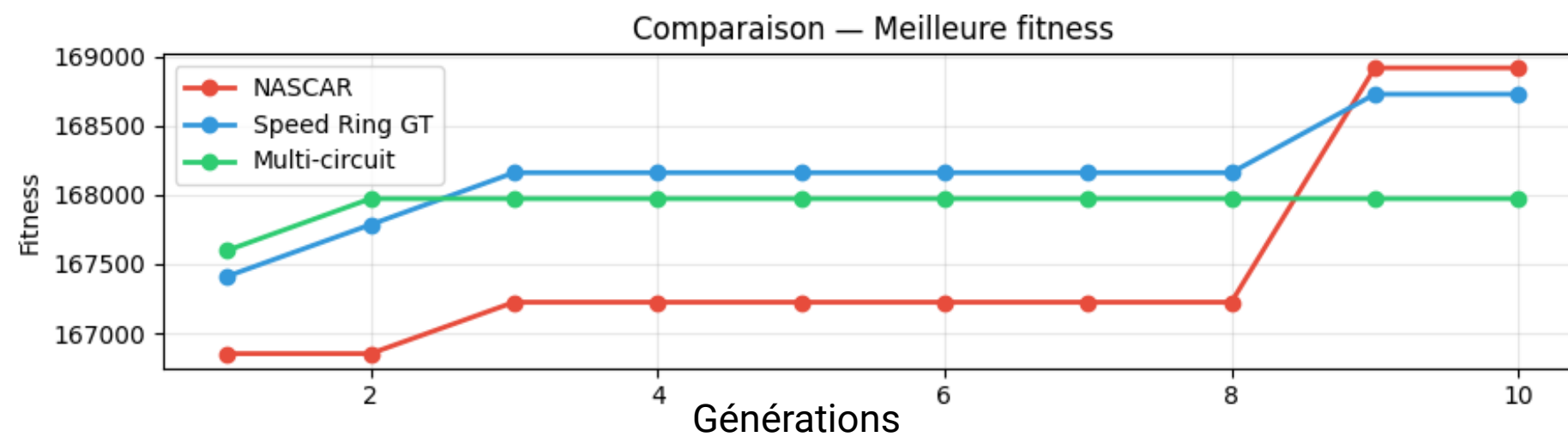
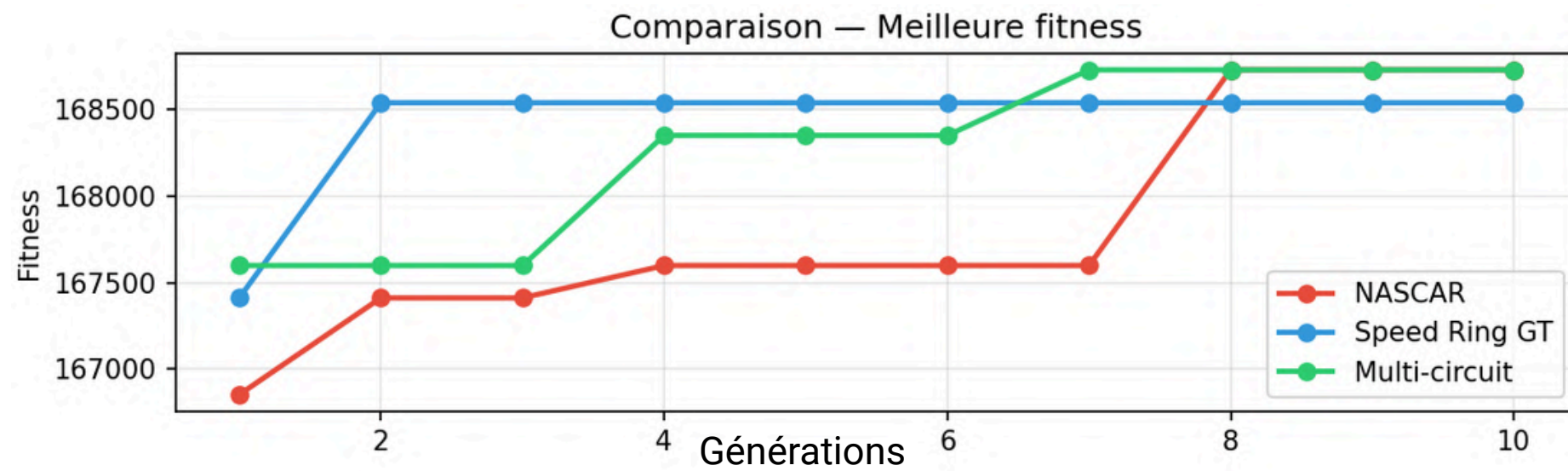


Constat :

- Apprentissage en multi-circuits plus performant dès le début
- Entraînement sur un seul circuit peu efficace

... Et le meilleur agent ?

Apprentissage : Mono VS Multi-circuits



Constat :

- Non généralisable
- Meilleur agent = hasard

Comparaison des différentes méthodes

[illegible]

Comparaison des différentes méthodes

	Q-learning	Algorithme génétique
Rapidité	Rapide (2-5s / agent)	Moyenne (10s / gen)
Généralisation (s'adapte à plusieurs circuits)	Moyenne/Faible (apprentissage presque total)	Bonne (comportement similaire à déjà entraîné)
		17/19

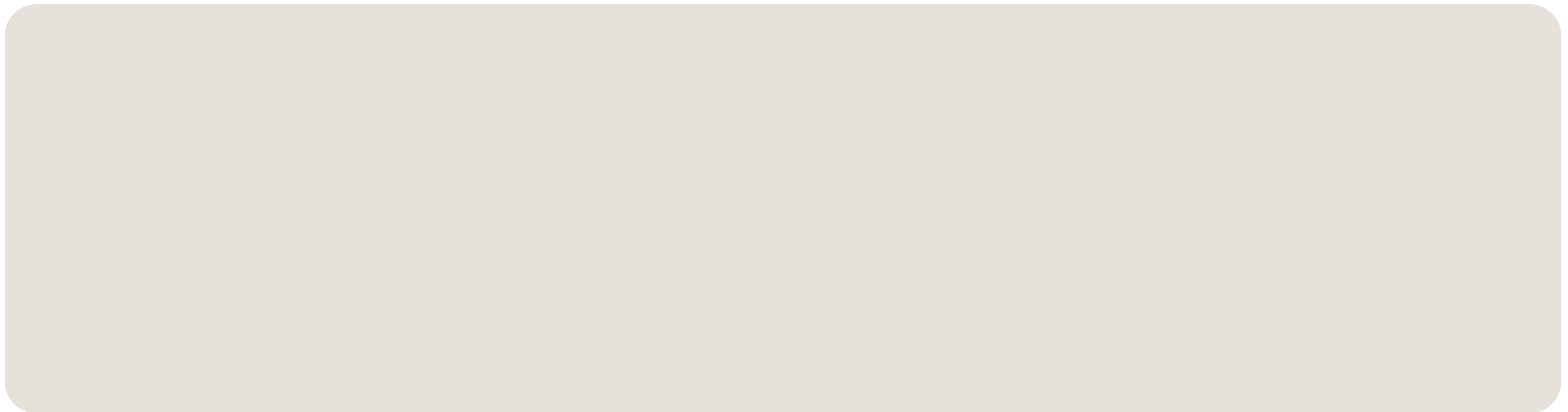
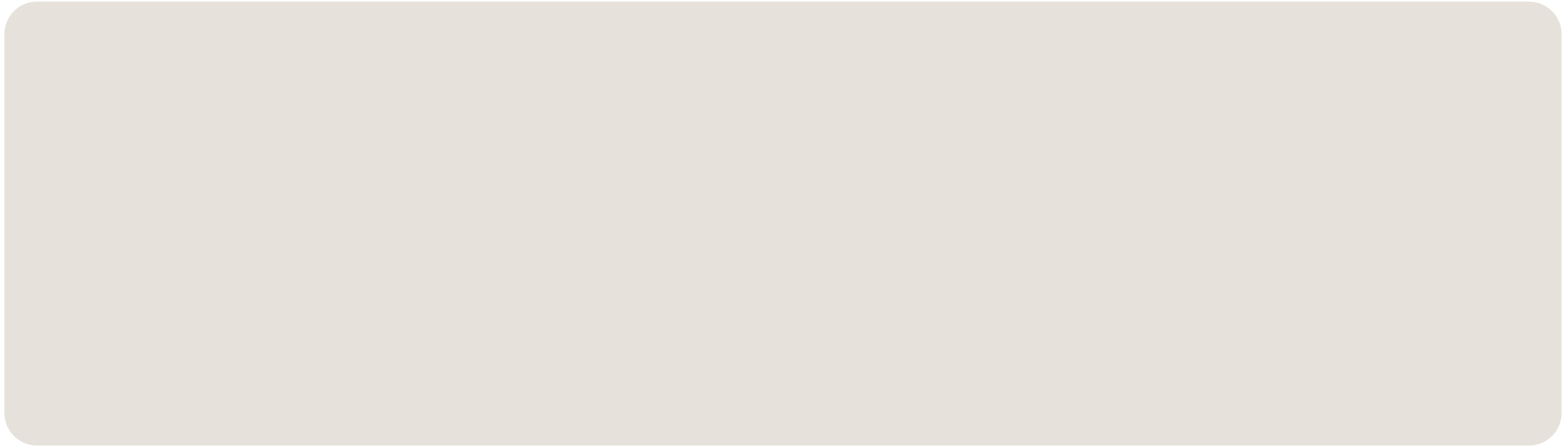
Comparaison des différentes méthodes

	Q-learnings	Algorithme génétique
Rapidité	Rapide (2-5s / agent)	Moyenne (10s / gen)
Généralisation (s'adapte à plusieurs circuits)	Moyenne/Faible (apprentissage presque total)	Bonne (comportement similaire à déjà entraîné)
Robustesse (évolution même avec un mauvais départ)	Faible (explosion du nombre d'agents nécessaires)	Robuste (grand pool d'agents)
Possibilité de réalisme	Très faible (actions discrètes)	Très bonne
		17/19

Comparaison des différentes méthodes

	Q-learning	Algorithme génétique
Rapidité	Rapide (2-5s / agent)	Moyenne (10s / gen)
Généralisation (s'adapte à plusieurs circuits)	Moyenne/Faible (apprentissage presque total)	Bonne (comportement similaire à déjà entraîné)
Robustesse (évolution même avec un mauvais départ)	Faible (explosion du nombre d'agents nécessaires)	Robuste (grand pool d'agents)
Possibilité de réalisme	Très faible (actions discrètes)	Très bonne
Complexité (temps de convergence)	Moyen (50/100 agents)	Moyen (5/7 générations)

Idées reçues et explications :



Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Avantages :

- Rapide
- Simple à comprendre
- Se rapproche très facilement de la réalité

Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Avantages :

- Rapide
- Simple à comprendre
- Se rapproche très facilement de la réalité

Inconvénients :

- Aucun apprentissage
- Non généralisable
- Impossibilité de dépasser l'idéal fixé

Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Avantages :

- Rapide
- Simple à comprendre
- Se rapproche très facilement de la réalité

Inconvénients :

- Aucun apprentissage
- Non généralisable
- Impossibilité de dépasser l'idéal fixé

Création d'un apprentissage mêlant génétique + Q-learnings :

Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Avantages :

- Rapide
- Simple à comprendre
- Se rapproche très facilement de la réalité

Inconvénients :

- Aucun apprentissage
- Non généralisable
- Impossibilité de dépasser l'idéal fixé

Création d'un apprentissage mêlant génétique + Q-learning :

Avantages hypothétiques :

- Simulation très rapide
- Convergence rapide

Idées reçues et explications :

Montrer la trajectoire optimale / stratégie parfaite à reproduire :

Avantages :

- Rapide
- Simple à comprendre
- Se rapproche très facilement de la réalité

Inconvénients :

- Aucun apprentissage
- Non généralisable
- Impossibilité de dépasser l'idéal fixé

Création d'un apprentissage mêlant génétique + Q-learnings :

Avantages hypothétiques :

- Simulation très rapide
- Convergence rapide

Inconvénients réels :

Perte d'une liberté de mouvement (actions discrètes) et donc aucun intérêt au génétique

Conclusion

Algorithme et méthode adapté pour le meilleur rendement:

Approche génétique

+

Entraînement multi-circuit

Mais, pour une approximation rapide :

Q-table

Annexes

```
class VectorizedCarEnv:
```

```
    def __init__(self, spawn_point, walls, track, track_width: float = 60.0,
        n_cars: int  = 1000, n_rays: int  = 9, device: str  = 'cuda',
        max_speed: float = None,  # ignoré - kept pour compatibilité interface
        collision_threshold: float = RAY_COLLISION_THRESHOLD,
        rain_mode: str = 'fixed',
        initial_rain: float = 0.0, initial_compound: int  = MEDIUM):
        self.device  = torch.device(device if torch.cuda.is_available() else 'cpu')
        self.n_cars   = n_cars
        self.n_rays   = n_rays
        self.track_width  = track_width
        self.collision_threshold = collision_threshold
```

```
    # -- Spawn -----
```

```
    if len(spawn_point) == 3:
        self.spawn_x, self.spawn_y, self.spawn_angle = spawn_point
    else:
        self.spawn_x, self.spawn_y = spawn_point
        self.spawn_angle = 0.0
```

```
    self.spawn_tensor = torch.tensor(
        [self.spawn_x, self.spawn_y],
        dtype=torch.float32, device=self.device
    )
```

```
    # -- Track -> tenseurs GPU pour collision -----
```

```
    self.track = track
    self._setup_track_tensors(track)
```

```
    # -- Murs -> tenseurs GPU pour raycasting -----
```

```
    starts, ends = [], []
    for w in walls:
        starts.append(w[0])
        ends.append(w[1])
    self.wall_starts = torch.tensor(starts, dtype=torch.float32, device=self.device)
    self.wall_ends = torch.tensor(ends, dtype=torch.float32, device=self.device)
```

```
    # -- Moteur physique -----
```

```
    self.physics = CarPhysics(device=str(self.device))
```

```
    # -- Systèmes -----
```

```
    self.tires = TireSystemGPU(n_cars, initial_compound, str(self.device))
    self.fuel = FuelSystemGPU(n_cars, device=str(self.device))
    self.rain = RainSystemGPU(
        mode=rain_mode, initial_intensity=initial_rain, device=str(self.device)
    )
```

```
    # -- Angles des rayons (fixes, calculés une fois) -----
```

```
    self.ray_angles = torch.linspace(
        -np.pi / 2, np.pi / 2, n_rays, device=self.device
    ) # (n_rays,)
```

```
# -- Tenseurs d'état - initialisés dans reset() -----
self.pos = torch.zeros((n_cars, 2), device=self.device)
self.speed = torch.zeros((n_cars, 1), device=self.device)
self.heading = torch.zeros((n_cars, 1), device=self.device)
self.slip_angle = torch.zeros((n_cars, 1), device=self.device)
self.alive = torch.ones(n_cars, dtype=torch.bool, device=self.device)
self.distances = torch.zeros((n_cars, n_rays), device=self.device)

# -- État pit stop -----
self.pit_timer = torch.zeros(n_cars, dtype=torch.int32, device=self.device)
self.pit_is_full = torch.zeros(n_cars, dtype=torch.bool, device=self.device)
self.in_pit_zone = torch.zeros(n_cars, dtype=torch.bool, device=self.device)

self.reset()
```

```
def _setup_track_tensors(self, track):
    """
    Convertit la géométrie du track en tenseurs GPU.
    Les sommets des polygones sont pré-shiftés (roll -1) une seule fois
    pour éviter de recalculer torch.roll à chaque step.
    """

    from tracks.track_geometry import AngularTrack, RectangularTrack

    if isinstance(track, AngularTrack):
        self.track_type = 'angular'

        outer = torch.tensor(track.outer, dtype=torch.float32, device=self.device)
        inner = torch.tensor(track.inner, dtype=torch.float32, device=self.device)

        # Pré-calcul des sommets suivants (edge i -> i+1)
        self.outer_poly  = outer
        self.outer_poly_next = torch.roll(outer, -1, dims=0)
        self.inner_poly  = inner
        self.inner_poly_next = torch.roll(inner, -1, dims=0)
```

```

elif isinstance(track, RectangularTrack):
self.track_type = 'rectangular'
ox_min, ox_max, oy_min, oy_max = track.outer_bounds
ix_min, ix_max, iy_min, iy_max = track.inner_bounds
self.outer_bounds_t = torch.tensor(
[ox_min, ox_max, oy_min, oy_max], dtype=torch.float32, device=self.device
)
self.inner_bounds_t = torch.tensor(
[ix_min, ix_max, iy_min, iy_max], dtype=torch.float32, device=self.device
)

else:
self.track_type = 'unknown'
print(
"[neuronal_env_v2] Warning Type de track inconnu - "
"fallback CPU pour la détection de collision (lent)."
)

```

```

def reset(self, randomize_spawn=False, random_range=10.0, random_angle=0.1):
"""
Remet toutes les voitures au départ.

Returns : observations Tensor (N, n_rays + 5)
"""

self.pos[:, 0] = self.spawn_x
self.pos[:, 1] = self.spawn_y

if randomize_spawn:
    self.pos += torch.randn((self.n_cars, 2), device=self.device) * random_range
    self.heading[:] = self.spawn_angle + \
        torch.randn((self.n_cars, 1), device=self.device) * random_angle
else:
    self.heading[:] = self.spawn_angle

self.speed.zero_()
self.slip_angle.zero_()
self.alive[:] = True
self.distances.zero_()
self.pit_timer.zero_()
self.pit_is_full.zero_()
self.in_pit_zone.zero_()
self.tires.reset()
self.fuel.reset()
self.rain.reset()

return self.get_observations()

```

```
def step(self, actions: torch.Tensor):
```

```
    """
```

```
    Avance d'un pas de temps.
```

```
    Args
```

```
    ----
```

```
    actions : Tensor (N, 4)
```

```
    [:, 0] steering dans [-1, 1]
```

```
    [:, 1] throttle dans [-1, 1] (négatif = frein)
```

```
    [:, 2] pit_tire dans [0, 1] (> PIT_THRESHOLD -> pit pneus)
```

```
    [:, 3] pit_full dans [0, 1] (> PIT_THRESHOLD -> pit complet)
```

```
    Returns
```

```
    -----
```

```
    observations : Tensor (N, n_rays + 5)
```

```
    rewards      : Tensor (N,)
```

```
    done         : Tensor (N,) bool
```

```
    """
```

```
    steering = actions[:, 0:1] # (N, 1)
```

```
    throttle = actions[:, 1:2] # (N, 1)
```

```
    sig_tire = actions[:, 2]   # (N,)
```

```
    sig_full = actions[:, 3]   # (N,)
```

```
# -- 1. Gestion pit stop -----
```

```
in_pit = self.pit_timer > 0 # (N,) - voitures actuellement au stand
```

```
# Décrémenter le timer des voitures au stand
```

```
self.pit_timer = (self.pit_timer - 1).clamp(min=0)
```

```
# Appliquer le pit stop quand le timer vient d'atteindre 0
```

```
just_finished = in_pit & (self.pit_timer == 0) # (N,)
```

```
if just_finished.any():
```

```
    self.tires.apply_pit_stop(just_finished)
```

```
    full_done = just_finished & self.pit_is_full
```

```
    if full_done.any():
```

```
        self.fuel.refuel(full_done)
```

```
        self.pit_is_full[just_finished] = False
```

```
# Déclencher un nouveau pit stop
```

```
# Condition : vivante + pas déjà au stand + signal > seuil
```

```
can_pit = self.alive & ~in_pit # (N,)
```

```
new_tire = can_pit & (sig_tire > PIT_THRESHOLD)
```

```
new_full = can_pit & (sig_full > PIT_THRESHOLD)
```

```
new_tire = new_tire & ~new_full # pit_full prioritaire
```

```
if new_tire.any():
```

```
    self.pit_timer[new_tire] = PIT_TIRE_DURATION
```

```
    self.pit_is_full[new_tire] = False
```

```
if new_full.any():
```

```
    self.pit_timer[new_full] = PIT_FULL_DURATION
```

```
    self.pit_is_full[new_full] = True
```

```

# -- 2. Physique -----

# Voitures actives = vivantes ET pas au stand
active = self.alive & ~in_pit # (N,)

rain = self.rain.intensity # float scalaire
grip = self.tires.grip_multiplier(rain) # (N, 1)
effective_throttle = throttle * self.fuel.fuel_throttle_factor() # (N, 1)

self.pos, self.speed, self.heading, self.slip_angle = self.physics.step(
self.pos, self.speed, self.heading, self.slip_angle,
active, steering, effective_throttle, grip )

# -- 3. Mise à jour des systèmes -----
self.tires.update_wear(self.speed, rain)
self.fuel.consume(self.speed)
self.rain.step()

# -- 4. Zone de pit -----
dist_to_spawn = torch.norm(
self.pos - self.spawn_tensor.unsqueeze(0), dim=1
) # (N,)
self.in_pit_zone = dist_to_spawn < PIT_ZONE_RADIUS # (N,) bool

# -- 5. Raycasting -----
self.distances = self._compute_lidar()

# -- 6. Détection de collision -----
ray_crash = (self.distances < self.collision_threshold).any(dim=1) # (N,)
track_crash = self._check_track_collision_gpu() # (N,)

```

```

# Les voitures au stand sont invulnérables
crashes = (ray_crash | track_crash) & ~in_pit
self.alive = self.alive & ~crashes

# -- 7. Observations et retour -----
observations = self.get_observations()
rewards = self.speed.squeeze(1) * self.alive.float()
dones = ~self.alive

return observations, rewards, dones

def _compute_lidar(self) -> torch.Tensor:
    """
    Calcule les distances lidar pour toutes les voitures.

    heading (N, 1) + ray_angles (n_rays,) -> broadcast (N, n_rays)
    Puis aplatissement -> (N×n_rays, 2) pour compute_ray_intersections.

    Returns : (N, n_rays) distances en pixels
    """
    from learnings.ray_casting.gpu_raycasting import compute_ray_intersections

    global_angles = self.heading + self.ray_angles # (N, n_rays) broadcast
    flat_angles = global_angles.view(-1) # (N×n_rays,)

    ray_dirs = torch.stack(
        [torch.cos(flat_angles), torch.sin(flat_angles)], dim=1
    ) # (N×n_rays, 2)
    ray_origins = self.pos.repeat_interleave(self.n_rays, dim=0) # (N×n_rays, 2)

```

```

dists = compute_ray_intersections(
ray_origins, ray_dirs,
self.wall_starts, self.wall_ends,
max_dist=3.0 * self.track_width
) # (N×n_rays,)

return dists.view(self.n_cars, self.n_rays) # (N, n_rays)

```

```

def _inside_polygon_gpu( self, points  : torch.Tensor, # (N, 2)
    poly  : torch.Tensor, # (M, 2)
    poly_next : torch.Tensor, # (M, 2) = roll(poly, -1) pré-calculé
) -> torch.Tensor:
    # Coordonnées des points : (N, 1)
    px = points[:, 0].unsqueeze(1) # (N, 1)
    py = points[:, 1].unsqueeze(1) # (N, 1)

    # Coordonnées des arêtes : (1, M)
    x1 = poly[:, 0].unsqueeze(0) # (1, M)
    y1 = poly[:, 1].unsqueeze(0) # (1, M)
    x2 = poly_next[:, 0].unsqueeze(0) # (1, M)
    y2 = poly_next[:, 1].unsqueeze(0) # (1, M)

    # Condition 1 : py est strictement entre y1 et y2
    cond1 = (y1 > py) != (y2 > py) # (N, M) bool

    # Abscisse de l'intersection du rayon horizontal y=py avec l'arête
    dy = y2 - y1 # (1, M)
    safe_dy = torch.where(dy.abs() < 1e-9, torch.full_like(dy, 1e-9), dy)
    xinters = x1 + (py - y1) * (x2 - x1) / safe_dy # (N, M)

    # Condition 2 : l'intersection est à droite de px
    cond2 = px < xinters # (N, M) bool

    # Nombre de traversées - impair = intérieur
    crossings = (cond1 & cond2).int().sum(dim=1) # (N,)
    return (crossings % 2) == 1 # (N,) bool

```

```

def _check_track_collision_gpu(self) -> torch.Tensor:
    if self.track_type == 'angular':
        in_outer = self._inside_polygon_gpu(
            self.pos, self.outer_poly, self.outer_poly_next
        )
        in_inner = self._inside_polygon_gpu(
            self.pos, self.inner_poly, self.inner_poly_next
        )
        # Sur le circuit = dans outer ET hors inner
        return ~(in_outer & ~in_inner) # (N,) bool

    elif self.track_type == 'rectangular':
        x = self.pos[:, 0]
        y = self.pos[:, 1]
        ox_min, ox_max, oy_min, oy_max = self.outer_bounds_t
        ix_min, ix_max, iy_min, iy_max = self.inner_bounds_t
        in_outer = (x >= ox_min) & (x <= ox_max) & (y >= oy_min) & (y <= oy_max)
        in_inner = (x >= ix_min) & (x <= ix_max) & (y >= iy_min) & (y <= iy_max)
        return ~(in_outer & ~in_inner)

    else:
        # Fallback CPU - affiché au __init__, ne devrait pas arriver
        collisions = torch.zeros(self.n_cars, dtype=torch.bool, device=self.device)
        pos_cpu = self.pos.cpu().numpy()
        for i in range(self.n_cars):
            if self.alive[i] and not self.track.is_inside(*pos_cpu[i]):
                collisions[i] = True
        return collisions

def get_observations(self) -> torch.Tensor:
    max_range = 3.0 * self.track_width

    rays = (self.distances / max_range).clamp(0.0, 1.0) # (N, n_rays)
    speed = (self.speed / BASE_MAX_SPEED).clamp(0.0, 1.0) # (N, 1)
    wear = self.tires.wear.unsqueeze(1) # (N, 1)
    fuel = self.fuel.fuel.unsqueeze(1) # (N, 1)
    rain = torch.full(
        (self.n_cars, 1), self.rain.intensity, device=self.device
    ) # (N, 1)
    compound = (self.tires.compound.float() / N_COMPOUNDS).unsqueeze(1) # (N, 1)
    return torch.cat([rays, speed, wear, fuel, rain, compound], dim=1) # (N, n_rays+5)

# Render
def get_render_data(self) -> dict:
    return {
        'pos' : self.pos.cpu().numpy(),
        'angle' : self.heading.cpu().numpy(),
        'alive' : self.alive.cpu().numpy(),
        'speed' : self.speed.cpu().numpy(),
        'slip_angle' : self.slip_angle.cpu().numpy(),
        'in_pit_zone': self.in_pit_zone.cpu().numpy(),
        'pit_timer' : self.pit_timer.cpu().numpy() }

# Utilitaires
def get_alive_count(self) -> int:
    return self.alive.sum().item()

def is_all_dead(self) -> bool:
    return not self.alive.any()

```

```
def parse_args():
    parser = argparse.ArgumentParser(
        description="Entraînement AG v2 — physique réaliste (pneus / carburant / pluie)"
    )

    # -- Entraînement -----
    parser.add_argument('--generations', type=int, default=100)
    parser.add_argument('--population', type=int, default=1000)
    parser.add_argument('--n_rays', type=int, default=9)
    parser.add_argument('--save_every', type=int, default=10, help="Sauvegarder tous les N générations")
    parser.add_argument('--device', type=str, default='cuda', choices=['cuda', 'cpu'])
    parser.add_argument('--checkpoint', type=str, default=None, help="Chemin vers un checkpoint à reprendre")
    parser.add_argument('--frequency_showgen', type=int, default=-1, help="Afficher le rendu toutes les N générations (-1 = jamais)")

    # -- Circuits -----
    parser.add_argument('--random_train', type=int, default=1, help="Nombre de circuits (triés par difficulté). 0 = utiliser --circuit")
    parser.add_argument('--circuit', type=str, nargs='+', default=['nascar'], choices=['nascar', 'rectangle', 'speed_ring_gt', 'w_track'], help="Circuit(s) à utiliser si --random_train 0")
    parser.add_argument('--nb_laps', type=int, default=1, help="Tours pour terminer une génération (prioritaire sur nb_steps)")
    parser.add_argument('--nb_steps', type=int, default=2000, help="Steps max par génération (fallback si nb_laps non atteint)")

    # -- Mutation -----
    parser.add_argument('--mutation_start', type=float, default=0.3)
    parser.add_argument('--mutation_end', type=float, default=0.05)
    parser.add_argument('--mutation_decay', type=float, default=0.975)
```

```
# -- Systèmes physiques -----
parser.add_argument('--rain_mode', type=str, default='fixed', choices=['fixed', 'dynamic', 'preset'], help="Mode d'évolution de la pluie")
parser.add_argument('--initial_rain', type=float, default=0.0, help="Intensité de pluie initiale [0.0=sec, 1.0=déluge]")
parser.add_argument('--compound', type=str, default='medium', choices=list(_COMPOUND_MAP.keys()), help="Composé de départ pour tous les agents")

# -- Curriculum learning (savoir quand on accède aux nouvelles actions) -----
parser.add_argument('--phase2_gen', type=int, default=10, help="Génération de déclenchement de la phase 2 (-1 = désactivé)")
parser.add_argument('--phase2_fitness', type=float, default=None, help="Seuil avg_fitness pour déclencher la phase 2 (None = désactivé)")

return parser.parse_args()
```

```
def main():
    args = parse_args()

    # -- 1. GPU -----
    if args.device == 'cuda' and not torch.cuda.is_available():
        print("CUDA non disponible, passage en CPU")
        args.device = 'cpu'

    if args.device == 'cuda':
        print(f"GPU : {torch.cuda.get_device_name(0)}")
        print(f"VRAM : {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")

    initial_compound = _COMPOUND_MAP[args.compound]
    phase2_gen = args.phase2_gen if args.phase2_gen >= 0 else None

    print(
        f"\nSystèmes : pluie={args.rain_mode} (init={args.initial_rain:.2f}) | "
        f"composé={args.compound.capitalize()}" )
```

```
print(
    f"Curriculum : phase 2 à gen≥{phase2_gen} "
    f"ou avg_fitness≥{args.phase2_fitness}\n"
)

# -- 2. Circuits -----
configs = load_circuits(
    n = args.random_train if args.random_train > 0 else None,
    names = args.circuit if args.random_train == 0 else None,
    n_cars = args.population,
    n_rays = args.n_rays,
    device = args.device,
    rain_mode = args.rain_mode,
    initial_rain = args.initial_rain,
    initial_compound = initial_compound,
)
```

```

if len(configs) == 1:
    cfg = configs[0]
    print(f"Circuit unique : {cfg['name']} (difficulté {cfg['difficulty']})")
else:
    print(f"{len(configs)} circuits chargés :")
    for c in configs:
        print(f" - {c['name']} (difficulté {c['difficulty']})")

cfg = configs[0]
env = cfg["env"]
checkpoints = cfg["checkpoints"]
walls = cfg["walls"]

print(f"Voitures : {args.population} | Rayons : {args.n_rays} | Checkpoints : {len(checkpoints)}")

# -- 3. FitnessTracker -----
fitness_tracker = FitnessTracker( checkpoints = checkpoints, spawn_point = (env.spawn_x, env.spawn_y, env.spawn_angle),
    n_cars = args.population, track_width = env.track_width, walls = walls, device = args.device)

# -- 4. PopulationManager -----
population_manager = PopulationManager( n_population = args.population, n_rays = args.n_rays, initial_mutation_rate = args.mutation_start,
    final_mutation_rate = args.mutation_end, mutation_decay = args.mutation_decay, device = args.device )

# -- 5. Checkpoint -----
if args.checkpoint:
    print(f"\nChargement checkpoint : {args.checkpoint}")
    population_manager.load_population_from_file(args.checkpoint)

```

```
# -- 6. TrainingLoop -----
```

```
training_loop = TrainingLoop( env = env, population_manager = population_manager,  
    fitness_tracker = fitness_tracker, frequency_show = args.frequency_showgen,  
    walls = walls, all_configs = configs, max_laps = args.nb_laps, max_steps = args.nb_steps,  
    phase2_gen = phase2_gen, phase2_fitness = args.phase2_fitness)
```

```
# -- 7. Dossier de sauvegarde -----
```

```
circuit_names = "_".join(c["name"] for c in configs)  
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")  
save_dir = f"checkpoints/{circuit_names}_{timestamp}"  
os.makedirs(save_dir, exist_ok=True)
```

```
config_path = os.path.join(save_dir, "config.txt")
```

```
with open(config_path, 'w') as f:
```

```
    f.write(f"Circuits: {circuit_names}\n")  
    f.write(f"Population: {args.population}\n")  
    f.write(f"Générations: {args.generations}\n")  
    f.write(f"Rayons: {args.n_rays}\n")  
    f.write(f"Mutation: {args.mutation_start} -> {args.mutation_end} (decay={args.mutation_decay})\n")  
    f.write(f"Device: {args.device}\n")  
    f.write(f"Pluie: {args.rain_mode} (init={args.initial_rain})\n")  
    f.write(f"Composé: {args.compound}\n")  
    f.write(f"Phase 2 gen: {phase2_gen}\n")  
    f.write(f"Phase 2 fitness: {args.phase2_fitness}\n")
```

```
print(f"\nSauvegardes dans : {save_dir}")
```

```
# -- 8. Entraînement -----
```

```
print(f"\nDÉBUT DE L'ENTRAÎNEMENT")
```

```
print("=" * 100 + "\n")
```

```
training_loop.train(  
    n_generations = args.generations,  
    save_every = args.save_every,  
    save_path = save_dir,  
)
```

```
# -- 9. Statistiques finales -----
```

```
print(f"\n{'='*100}")
```

```
print("ENTRAÎNEMENT TERMINÉ")
```

```
print(f"{'='*100}")
```

```
print(f"Fichiers sauvegardés dans : {save_dir}")
```

```
do_visu = input("\nAfficher les courbes de progression ? (Y/N) : ").strip().upper()
```

```
if do_visu == "Y":
```

```
    from learnings.genetic_algorithm.show_improve import main as show_main  
    show_main(save_dir)
```

```
if __name__ == "__main__":  
    main()
```

class PopulationManager:

pop_manager_physic.py

```
def __init__(self, n_population: int = 1000, n_rays: int = 9,
n_system_obs: int = N_SYSTEM_OBS, hidden_size1: int = 32,
hidden_size2: int = 16, initial_mutation_rate: float = 0.3,
final_mutation_rate: float = 0.001, mutation_decay: float = 0.995,
mutation_strength: float = 0.3, device: str = 'cuda'):
self.n_population = n_population
self.n_rays = n_rays
self.n_system_obs = n_system_obs
self.hidden_size1 = hidden_size1
self.hidden_size2 = hidden_size2
self.device = torch.device(device if torch.cuda.is_available() else 'cpu')

self.mutation_rate = initial_mutation_rate
self.final_mutation_rate = final_mutation_rate
self.mutation_decay = mutation_decay
self.mutation_strength = mutation_strength

self.population = VectorizedNeuralPopulation(
n_agents = n_population, n_rays = n_rays, n_system_obs = n_system_obs,
hidden_size1 = hidden_size1, hidden_size2 = hidden_size2,
device = str(self.device) )

self.generation = 0
self.best_fitness_history = []
self.avg_fitness_history = []
```

```
def get_actions(self, observations: torch.Tensor) -> torch.Tensor:
with torch.no_grad():
return self.population.forward_vectorized(observations)

def select_top_percent( self, fitness_scores : torch.Tensor,
top_percent : float = 0.1, # Le top 10% ):
n_keep = max(2, int(self.n_population * top_percent))
sorted_idx = torch.argsort(fitness_scores, descending=True)
top_idx = sorted_idx[:n_keep]
elite_genomes = self.population.genomes[top_idx].clone()

print(
f"Sélection : top {n_keep} agents "
f"(fitness max : {fitness_scores[top_idx[0]].item():.2f})\n"
)
return elite_genomes, top_idx
```

```

def reproduce(self, elite_genomes: torch.Tensor):
    """Crossover + mutation entièrement vectorisés - zéro boucle Python."""
    n_elite = len(elite_genomes)
    genome_size = elite_genomes.shape[1]
    device = self.device

    print(f"Reproduction : {n_elite} parents -> {self.n_population} enfants\n")

    new_genomes = torch.empty((self.n_population, genome_size), device=device)
    new_genomes[:n_elite] = elite_genomes # élitisme

    n_children = self.n_population - n_elite
    if n_children > 0:
        idx1 = torch.randint(n_elite, (n_children,), device=device)
        idx2 = torch.randint(n_elite, (n_children,), device=device)
        p1 = elite_genomes[idx1]
        p2 = elite_genomes[idx2]

        # Crossover uniforme
        mask = torch.rand(n_children, genome_size, device=device) < 0.5
        children = torch.where(mask, p1, p2)

        # Mutation
        mut_mask = torch.rand_like(children) < self.mutation_rate
        noise = (torch.rand_like(children) * 2.0 - 1.0) * self.mutation_strength
        children += noise * mut_mask.float()

        new_genomes[n_elite:] = children

    self.population.genomes = new_genomes

```

```

def evolve(self, fitness_scores: torch.Tensor) -> dict:
    elite_genomes, _ = self.select_top_percent(fitness_scores, top_percent=0.1)
    self.reproduce(elite_genomes)

    self.mutation_rate = max( self.final_mutation_rate,
        self.mutation_rate * self.mutation_decay )
    self.generation += 1

    best = fitness_scores.max().item()
    avg = fitness_scores.mean().item()
    self.best_fitness_history.append(best)
    self.avg_fitness_history.append(avg)

    return {
        'generation' : self.generation,
        'best_fitness': best,
        'avg_fitness' : avg,
        'mutation_rate': self.mutation_rate,
        'elite_size' : int(self.n_population * 0.1),
    }

def save_best_agent(self, filepath: str, fitness_scores: torch.Tensor):
    best_idx = torch.argmax(fitness_scores).item()
    best_agent = self.population.get_agent(best_idx)
    best_agent.save_to_file(filepath)
    print(
        f"Meilleur agent sauvegardé : {filepath} "
        f"(fitness : {fitness_scores[best_idx].item():.2f})\n"
    )

```

```

def save_population(self, filepath: str):
    torch.save({
        'genomes'      : self.population.genomes,
        'generation'    : self.generation,
        'mutation_rate' : self.mutation_rate,
        'best_fitness_history': self.best_fitness_history,
        'avg_fitness_history' : self.avg_fitness_history,
        'n_rays'        : self.n_rays,
        'n_system_obs'   : self.n_system_obs,
        'hidden_size1'   : self.hidden_size1,
        'hidden_size2'   : self.hidden_size2,
    }, filepath)
    print(f"Population sauvegardée : {filepath}")

def load_population_from_file(self, filepath: str):
    ckpt = torch.load(filepath, map_location=self.device)
    if 'genomes' not in ckpt:
        print("Format invalide (clé 'genomes' manquante)")
        return

    self.population.genomes = ckpt['genomes'].to(self.device)
    self.generation = ckpt.get('generation', 0)
    self.mutation_rate = ckpt.get('mutation_rate', self.mutation_rate)
    self.n_system_obs = ckpt.get('n_system_obs', N_SYSTEM_OBS)
    self.hidden_size1 = ckpt.get('hidden_size1', 32)
    self.hidden_size2 = ckpt.get('hidden_size2', 16)
    print(f"Population chargée : génération {self.generation}")

```

```

def get_statistics(self) -> dict:
    return {
        'generation'    : self.generation,
        'best_fitness_history': self.best_fitness_history,
        'avg_fitness_history' : self.avg_fitness_history,
        'current_mutation_rate': self.mutation_rate,
    }

```

```

def reset_population(self):
    self.population = VectorizedNeuralPopulation(
        n_agents = self.n_population,
        n_rays   = self.n_rays,
        n_system_obs = self.n_system_obs,
        hidden_size1 = self.hidden_size1,
        hidden_size2 = self.hidden_size2,
        device     = str(self.device),
    )
    self.generation = 0
    self.mutation_rate = 0.3
    print("Population réinitialisée")

```

```

class TrainingLoop:
    """
    Boucle d'entraînement avec curriculum learning en 2 phases.
    """

    def __init__(
        self,
        env,
        population_manager,
        fitness_tracker,
        frequency_show: int = 0,
        walls: list = None,
        all_configs: list = None,
        max_laps: int | None = None,
        max_steps: int | None = None,
        phase2_gen: int | None = 10,
        phase2_fitness: float | None = None,
    ):
        self.env = env
        self.pop_manager = population_manager
        self.fitness_tracker = fitness_tracker
        self.frequency_show = frequency_show
        self.walls = walls
        self.renderer = VectorizedRenderer(show_dead=True)
            if frequency_show != 0 else None

    # Multi-circuit
    self.all_configs = all_configs if all_configs and len(all_configs) > 1 else None
    self._circuit_idx = 0

```

```

# Conditions d'arrêt de génération
self.max_laps = max_laps
self.max_steps = max_steps

# Curriculum learning
self.phase = 1 # démarre toujours en phase 1
self.phase2_gen = phase2_gen # génération de déclenchement (None = désactivé)
self.phase2_fitness = phase2_fitness # seuil fitness (None = désactivé)

def _apply_action_mask(self, actions: torch.Tensor) -> torch.Tensor:

    if self.phase == 1:
        # Masque total : aucun pit possible
        actions[:, 2:4] = 0.0

    elif self.phase == 2:
        # Masque partiel : pit interdit hors zone
        # env.in_pit_zone est un Tensor (N,) bool mis à jour dans neuronal_env_v2.step()
        if hasattr(self.env, 'in_pit_zone'):
            out_of_pit = ~self.env.in_pit_zone    # (N,) bool
            actions[out_of_pit, 2:4] = 0.0
        else:
            # Sécurité : si l'env ne définit pas in_pit_zone, on masque tout
            actions[:, 2:4] = 0.0

    return actions

```

```

def _check_phase_transition(self, stats: dict):
    """
    Vérifie si les critères de passage en phase 2 sont atteints.
    Affiche un message une seule fois au moment de la transition.
    """

    if self.phase != 1:
        return # déjà en phase 2, rien à faire

    triggered = False
    reason = ""

    if self.phase2_gen is not None:
        if self.pop_manager.generation >= self.phase2_gen:
            triggered = True
            reason = f"génération {self.pop_manager.generation} ≥ {self.phase2_gen}"

    if self.phase2_fitness is not None and not triggered:
        if stats.get('avg_fitness', 0.0) >= self.phase2_fitness:
            triggered = True
            reason = (
                f"avg_fitness {stats['avg_fitness']:.2f} "
                f"≥ seuil {self.phase2_fitness:.2f}"
            )

    if triggered:
        self.phase = 2
        print(f"\n{'='*60}")
        print(f" PHASE 2 DÉBLOQUÉE - Pit stops activés ({reason})")
        print(f"{'='*60}\n")

```

```

def _rotate_circuit(self):
    cfg = self.all_configs[self._circuit_idx]
    self._circuit_idx = (self._circuit_idx + 1) % len(self.all_configs)
    self.env = cfg["env"]
    self.walls = cfg["walls"]
    new_checkpoints = cfg["checkpoints"]
    n_new = len(new_checkpoints)

    self.fitness_tracker.checkpoints = torch.tensor(
        new_checkpoints, dtype=torch.float32,
        device=self.fitness_tracker.device
    )
    self.fitness_tracker.n_checkpoints = n_new
    self.fitness_tracker.spawn_point = (
        self.env.spawn_x, self.env.spawn_y, self.env.spawn_angle
    )
    self.fitness_tracker._compute_finish_line(
        (self.env.spawn_x, self.env.spawn_y, self.env.spawn_angle),
        cfg["walls"]
    )
    self.fitness_tracker.checkpoint_status = torch.zeros(
        (self.fitness_tracker.n_cars, n_new),
        dtype=torch.bool, device=self.fitness_tracker.device
    )
    self.fitness_tracker.treshold = self.env.track_width

    print(f" Circuit : {cfg['name']} (difficulté {cfg['difficulty']})")

```

```

def _is_generation_done(self, step: int) -> bool:
    if not self.env.alive.any():
        return True
    if self.max_laps is not None:
        if bool((self.fitness_tracker.laps_completed >= self.max_laps).any()):
            return True
    if self.max_steps is not None:
        if step >= self.max_steps:
            return True
    return False

def run_generation(self, render: bool = False, generation: int = 0) -> dict | None:
    observations = self.env.reset()
    self.fitness_tracker.reset()
    step = 0
    while not self._is_generation_done(step):

        # Forward pass réseau
        actions = self.pop_manager.get_actions(observations)

        # -- Curriculum : masquage des pit stops selon la phase -----
        actions = self._apply_action_mask(actions)

        # Step environnement
        observations, rewards, dones = self.env.step(actions)

        self.fitness_tracker.update(positions = self.env.pos,
        speeds = self.env.speed,
        alive_mask = self.env.alive)

```

```

    if render:
        render_data = self.env.get_render_data()
        still_running = self.renderer.render_step(generation, render_data,
            self.walls, self.fitness_tracker.get_render_finish_line())
    if not still_running:
        return None

    step += 1

    best_laps = int(self.fitness_tracker.laps_completed.max().item())
    if best_laps > 0:
        print(f"Meilleur tour bouclé : {best_laps} (sur {step} steps)")
    else:
        print(f"Génération terminée après {step} steps (aucun tour complet)")

    fitness_scores = self.fitness_tracker.compute_fitness()
    stats = self.pop_manager.evolve(fitness_scores)
    stats.update(self.fitness_tracker.get_statistics())

    # -- Vérification transition de phase -----
    self._check_phase_transition(stats)

    return stats

```

```

def train(
    self,
    n_generations : int = 100,
    save_every : int = 10,
    save_path : str = "checkpoints",
):
    os.makedirs(save_path, exist_ok=True)
    print(
        f"Début : {n_generations} générations | "
        f"Phase 2 : gen≥{self.phase2_gen} ou fitness≥{self.phase2_fitness}\n"
        + "=" * 100
    )

    for gen in range(n_generations):
        print(f"\nGÉNÉRATION {gen + 1}/{n_generations} [Phase {self.phase}]")

        if self.all_configs is not None:
            self._rotate_circuit()

        render = (
            self.frequency_show != 0
            and (gen + 1) % self.frequency_show == 0
        )
        stats = self.run_generation(render=render, generation=gen)

```

```

if stats is None:
    print("Fenêtre fermée, arrêt de l'entraînement.")
    return

```

```

print(
    f"Best={stats['best_fitness']:.2f} | "
    f"Avg={stats['avg_fitness']:.2f} | "
    f"Mutation={stats['mutation_rate']:.1%} | "
    f"Meilleurs tours={stats['max_laps']}"
)
print("-" * 60)

```

```

if (gen + 1) % save_every == 0:
    self.pop_manager.save_population(
        os.path.join(save_path, f"gen_{gen + 1}.pt")
    )

```

```

class NeuralAgent(nn.Module):
    """
    Réseau MLP pour contrôler une voiture avec gestion des systèmes.
    """

    def __init__( self, n_rays    : int = 9, n_system_obs : int = N_SYSTEM_OBS,
                    hidden_size1 : int = 32, hidden_size2 : int = 16,
                    device      : str = 'cuda' ):
        super(NeuralAgent, self).__init__()

        self.n_rays    = n_rays
        self.n_system_obs = n_system_obs
        self.device     = device

        input_size = n_rays + 1 + n_system_obs # rayons + vitesse + systèmes

        self.network = nn.Sequential(
            nn.Linear(input_size, hidden_size1), nn.ReLU(),
            nn.Linear(hidden_size1, hidden_size2), nn.ReLU(),
            nn.Linear(hidden_size2, N_ACTIONS) # 4 sorties)

        self._initialize_weights()

    def _initialize_weights(self):
        for layer in self.network:
            if isinstance(layer, nn.Linear):
                nn.init.xavier_uniform_(layer.weight)
                nn.init.zeros_(layer.bias)

```

neural_network_physics.py

```

def forward(self, observations: torch.Tensor) -> torch.Tensor:
    """
    Propagation avant.
    """

    raw = self.network(observations)      # (batch, 4)

    steering = torch.tanh(raw[:, 0:1])    # [-1, 1]
    throttle = torch.tanh(raw[:, 1:2])    # [-1, 1] ← freinage possible
    pit_tire = torch.sigmoid(raw[:, 2:3])  # [0, 1]
    pit_full = torch.sigmoid(raw[:, 3:4])  # [0, 1]

    return torch.cat([steering, throttle, pit_tire, pit_full], dim=1) # (batch, 4)

def get_genome(self) -> torch.Tensor:
    """Extrait tous les poids sous forme de vecteur 1D."""
    return torch.cat([p.data.view(-1) for p in self.parameters()])

def set_genome(self, genome: torch.Tensor):
    """Injecte un génome (vecteur 1D) dans le réseau."""
    offset = 0
    for param in self.parameters():
        numel = param.numel()
        param.data = genome[offset:offset + numel].view(param.shape)
        offset += numel

```

```
def mutate(self, mutation_rate: float = 0.3, mutation_strength: float = 0.1):
    """Mutation aléatoire uniforme des poids."""
    with torch.no_grad():
        for param in self.parameters():
            mask = torch.rand_like(param) < mutation_rate
            noise = torch.rand_like(param) * 2 * mutation_strength - mutation_strength
            param.data += noise * mask.float()
```

```
def clone(self) -> 'NeuralAgent':
    c = NeuralAgent(
        n_rays=self.n_rays, n_system_obs=self.n_system_obs, device=self.device
    ).to(self.device)
    c.load_state_dict(self.state_dict())
    return c
```

```
def crossover(self, other: 'NeuralAgent', crossover_rate: float = 0.5) -> 'NeuralAgent':
    child = NeuralAgent(
        n_rays=self.n_rays, n_system_obs=self.n_system_obs, device=self.device
    ).to(self.device)
    g1 = self.get_genome()
    g2 = other.get_genome()
    mask = torch.rand_like(g1) < crossover_rate
    child.set_genome(torch.where(mask, g1, g2))
    return child
```

```
def save_to_file(self, filepath: str):
    torch.save(self.state_dict(), filepath)
```

```
def load_from_file(self, filepath: str):
    self.load_state_dict(torch.load(filepath))
```

```

class VectorizedNeuralPopulation:
    """
    Population entière gérée en un seul forward GPU via bmm.
    """

    def __init__( self, n_agents : int = 1000, n_rays : int = 9,
        n_system_obs : int = N_SYSTEM_OBS, hidden_size1 : int = 32,
        hidden_size2 : int = 16, device : str = 'cuda' ):
        self.n_agents = n_agents
        self.n_rays = n_rays
        self.n_system_obs = n_system_obs
        self.h1 = hidden_size1
        self.h2 = hidden_size2
        self.device = torch.device(device if torch.cuda.is_available() else 'cpu')

        # Taille du génome calculée depuis un modèle de référence
        ref = NeuralAgent(n_rays, n_system_obs, hidden_size1, hidden_size2).to(self.device)
        self.genome_size = sum(p.numel() for p in ref.parameters())
        self.input_size = n_rays + 1 + n_system_obs

        # Génomes initiaux (distribution gaussienne centrée, std=0.5)
        self.genomes = torch.randn(
            n_agents, self.genome_size, device=self.device
        ) * 0.5

def forward_vectorized(self, observations: torch.Tensor) -> torch.Tensor:
    """
    Forward pass pour toute la population en une seule passe GPU.
    """

    N = self.n_agents
    i = self.input_size
    h1 = self.h1
    h2 = self.h2

    # -- Découpage des génomes en matrices de poids -----
    offset = 0

    W1 = self.genomes[:, offset:offset + h1*i ].view(N, h1, i ); offset += h1 * i
    b1 = self.genomes[:, offset:offset + h1 ].view(N, h1, 1 ); offset += h1
    W2 = self.genomes[:, offset:offset + h2*h1].view(N, h2, h1); offset += h2 * h1
    b2 = self.genomes[:, offset:offset + h2 ].view(N, h2, 1 ); offset += h2
    W3 = self.genomes[:, offset:offset + 4*h2 ].view(N, 4, h2); offset += 4 * h2
    b3 = self.genomes[:, offset:offset + 4 ].view(N, 4, 1 )

    # -- Forward pass vectorisé -----
    x = observations.unsqueeze(-1) # (N, input, 1)
    x = torch.relu(torch.bmm(W1, x) + b1) # (N, h1, 1)
    x = torch.relu(torch.bmm(W2, x) + b2) # (N, h2, 1)
    x = torch.bmm(W3, x) + b3 # (N, 4, 1)
    x = x.squeeze(-1) # (N, 4)

```

```

# -- Activations par sortie -----
steering = torch.tanh(x[:, 0:1]) # (N, 1) [-1, 1]
throttle = torch.tanh(x[:, 1:2]) # (N, 1) [-1, 1]
pit_tire = torch.sigmoid(x[:, 2:3]) # (N, 1) [0, 1]
pit_full = torch.sigmoid(x[:, 3:4]) # (N, 1) [0, 1]

return torch.cat([steering, throttle, pit_tire, pit_full], dim=1) # (N, 4)

def get_agent(self, index: int) -> NeuralAgent:
    agent = NeuralAgent(
        self.n_rays, self.n_system_obs, self.h1, self.h2
    ).to(self.device)
    agent.set_genome(self.genomes[index])
    return agent

def set_agent(self, index: int, agent: NeuralAgent):
    self.genomes[index] = agent.get_genome()

def mutate_all(self, mutation_rate: float = 0.1, mutation_strength: float = 0.3):
    mask = torch.rand_like(self.genomes) < mutation_rate
    noise = (torch.rand_like(self.genomes) * 2.0 - 1.0) * mutation_strength
    self.genomes += noise * mask.float()

```

```

def save_population(self, filepath: str):
    torch.save({
        'genomes' : self.genomes,
        'n_agents' : self.n_agents,
        'n_rays' : self.n_rays,
        'n_system_obs': self.n_system_obs,
        'h1' : self.h1,
        'h2' : self.h2,
    }, filepath)

def load_population(self, filepath: str):
    data = torch.load(filepath, map_location=self.device)
    self.genomes = data['genomes'].to(self.device)
    self.n_agents = data['n_agents']
    self.n_rays = data['n_rays']
    self.n_system_obs = data.get('n_system_obs', N_SYSTEM_OBS)
    self.h1 = data.get('h1', 32)
    self.h2 = data.get('h2', 16)

```

```
def _load_module(path: Path):
    module_name = f"_circuit_dyn_{path.stem}"
    if module_name in sys.modules:
        return sys.modules[module_name]
    spec = importlib.util.spec_from_file_location(module_name, path)
    module = importlib.util.module_from_spec(spec)
    try:
        spec.loader.exec_module(module)
    except Exception as e:
        print(f"[circuits_loader_v2] Impossible de charger {path.name} : {e}")
        return None
    sys.modules[module_name] = module
    return module
```

circuit_loader.py

```
def _validate(module, path: Path) -> bool:
    missing = [fn for fn in _REQUIRED if not hasattr(module, fn)]
    if missing:
        print(
            f"[circuits_loader_v2] {path.name} ignoré — "
            f"fonctions manquantes : {' '.join(missing)}"
        )
        return False
    return True
```

```
def _build_config( module, n_cars: int, n_rays: int, device: str,
    rain_mode: str, initial_rain : float, initial_compound: int ) -> dict | None:
    name = module.get_name()
    try:
        walls  = module.get_walls()
        spawn  = module.get_spawn()
        track_width = module.get_width()
        checkpoints = module.get_checkpoints()
        track  = module.get_track_geo()
    except Exception as e:
        print(f"[circuits_loader_v2] Erreur lecture '{name}' : {e}")
        return None
```

```
try:
    env = VectorizedCarEnv(
        spawn_point  = spawn,
        walls        = walls,
        track         = track,
        track_width   = track_width,
        n_cars        = n_cars,
        n_rays        = n_rays,
        device        = device,
        rain_mode     = rain_mode,
        initial_rain  = initial_rain,
        initial_compound = initial_compound,
    )
except Exception as e:
    print(f"[circuits_loader_v2] VectorizedCarEnv a échoué pour '{name}' : {e}")
    return None
```

```

return {
    "name" : name,
    "difficulty" : module.get_difficulty(),
    "env" : env,
    "checkpoints" : checkpoints,
    "walls" : walls,
}

def load_circuits( tracks_dir  : str | Path = "tracks",
    n: int | None = None, names: list | None = None,
    n_cars: int = 1000, n_rays: int = 9, device: str = 'cuda',
    rain_mode: str = 'fixed', initial_rain: float = 0.0,
    initial_compound : int = MEDIUM, ) -> list:
    """
    Charge tous les circuits valides et retourne une liste de configs.
    """

    tracks_path = Path(tracks_dir)
    circuit_files = sorted(tracks_path.glob("circuit_*.py"))

    if not circuit_files:
        raise FileNotFoundError(
            f"Aucun fichier circuit_*.py trouvé dans {tracks_path.resolve()}" )

modules = []
    for path in circuit_files:
        module = _load_module(path)
        if module and _validate(module, path):
            modules.append(module)

```

```

if not modules:
    raise RuntimeError("Aucun circuit valide n'a pu être chargé.")
modules.sort(key=lambda m: m.get_difficulty())

if names is not None:
    name_set = set(names)
    modules = [m for m in modules if m.get_name() in name_set]
    not_found = name_set - {m.get_name() for m in modules}
    if not_found:
        print(f"Circuits introuvables : {not_found}")
    elif n is not None:
        modules = modules[:n]

    configs = []
    for module in modules:
        cfg = _build_config(
            module, n_cars, n_rays, device,
            rain_mode, initial_rain, initial_compound
        )
        if cfg is not None:
            configs.append(cfg)

if not configs:
    raise RuntimeError("Aucun environnement n'a pu être construit.")

print(
    f"Circuit loader: {len(configs)} circuit(s) prêt(s) : "
    + ", ".join(f"{c['name']}(diff={c['difficulty']})" for c in configs)
)
return configs

```

```
def get_name():  
    return "w_track"
```

circuit_w_track.py

```
def get_difficulty():  
    return 4
```

```
def get_spawn():  
    return 9 * 31.400, 9 * 27.457, 0.0
```

```
def _get_centerline():  
    line = [ (9 * 31.400, 9 * 27.457),  
    ...  
    (9 * 19.200, 9 * 21.514),  
    (9 * 24.400, 9 * 26.314)]  
    return line
```

```
def _build_walls_width():  
    """Construction unique — partagée entre get_walls"""  
    control_points = _get_centerline()  
    centerline = catmull_rom_spline(control_points)  
    outer, inner, width = generate_walls(centerline)  
    return outer, inner, width
```

```
def get_width():  
    _, _, width = _build_walls_width()  
    return width
```

```
def get_walls():  
    outer, inner, _ = _build_walls_width()  
    walls = [(inner[i], inner[i+1]) for i in range(len(inner) - 1)]  
    walls += [(outer[i], outer[i+1]) for i in range(len(outer) - 1)]  
    return walls
```

```
def get_checkpoints():  
    outer, inner, _ = _build_walls_width()  
    centerline = compute_centerline(outer, inner)  
    return centerline[::20]
```

```
def get_track_geo():  
    outer, inner, _ = _build_walls_width()  
    outer, inner, _ = _build_walls_width()
```

```
# outer et inner sont inversés sur ce circuit — on les swap  
outer_closed = inner + [inner[0]]  
inner_closed = outer + [outer[0]]
```

```
return AngularTrack(outer_closed, inner_closed)
```

def catmull_rom_segment(p0, p1, p2, p3, n_points=20): Catmull_Rom_geometry.py

```
"""
Génère n_points sur la spline Catmull-Rom entre p1 et p2
"""

p0, p1, p2, p3 = map(np.array, (p0, p1, p2, p3))
points = []

for t in np.linspace(0, 1, n_points):
    t2 = t * t
    t3 = t2 * t

    point = 0.5 * (
        (2 * p1)
        + (-p0 + p2) * t
        + (2*p0 - 5*p1 + 4*p2 - p3) * t2
        + (-p0 + 3*p1 - 3*p2 + p3) * t3
    )
    points.append(tuple(point))

return points
```

```
def catmull_rom_spline(points, n_points_per_segment=20, closed=True):
    """
    Génère une spline Catmull-Rom complète
    """

    if closed:
        pts = [points[-1]] + points + [points[0], points[1]]
    else:
        pts = [points[0]] + points + [points[-1]]

    curve = []

    for i in range(1, len(pts) - 2):
        seg = catmull_rom_segment(
            pts[i-1],
            pts[i],
            pts[i+1],
            pts[i+2],
            n_points_per_segment )
        curve.extend(seg)

    return curve
```

```
def compute_tangents(centerline: np.ndarray) -> np.ndarray:
    """
    Calcule les vecteurs tangents par différences centrées.
    """
    N = len(centerline)
    tangents = np.zeros_like(centerline)

    for i in range(N):
        p_prev = centerline[i - 1]
        p_next = centerline[(i + 1) % N] # périodique (circuit fermé)
        tangents[i] = p_next - p_prev

    return tangents
```

```
def compute_normals(tangents) -> np.ndarray:
    """
    Calcule les normales unitaires à partir des tangentes.
    """
    normals = np.zeros_like(tangents)

    for i, t in enumerate(tangents):
        norm = np.linalg.norm(t)
        if norm == 0:
            raise ValueError(f"Tangente nulle au point {i}")
        normals[i] = np.array([-t[1], t[0]]) / norm

    return normals
```

wall_from_center.py

```
def generate_walls(centerline: np.ndarray, width: float):
    """
    Génère les murs gauche et droit à partir de la ligne centrale.
    """
    tangents = compute_tangents(centerline)
    normals = compute_normals(tangents)

    left_wall = centerline + width * normals
    right_wall = centerline - width * normals

    return left_wall, right_wall

def plot_track(centerline, left_wall, right_wall):
    """
    Visualisation de contrôle géométrique.
    """
    plt.figure(figsize=(8, 8))
    plt.plot(centerline[:, 0], centerline[:, 1], 'k--', label="Ligne centrale")
    plt.plot(left_wall[:, 0], left_wall[:, 1], 'r', label="Mur gauche")
    plt.plot(right_wall[:, 0], right_wall[:, 1], 'b', label="Mur droit")
    plt.axis('equal')
    plt.legend()
    plt.title("Génération des murs du circuit")
    plt.show()
```

```

def parse_args():
    parser = argparse.ArgumentParser(
        description="Adaptation des agents pré-entraînés sur W-track"
    )

    parser.add_argument('--nascar', type=str, required=True)
    parser.add_argument('--gt', type=str, required=True)
    parser.add_argument('--multi', type=str, required=True)

    parser.add_argument('--n_adapt_gen', type=int, default=10)
    parser.add_argument('--population', type=int, default=1000)
    parser.add_argument('--n_rays', type=int, default=9)
    parser.add_argument('--nb_steps', type=int, default=2000)
    parser.add_argument('--nb_laps', type=int, default=1)
    parser.add_argument('--device', type=str, default='cuda', choices=['cuda', 'cpu'])
    parser.add_argument('--frequency_show', type=int, default = 0)

    parser.add_argument('--mutation_start', type=float, default=0.05)
    parser.add_argument('--mutation_end', type=float, default=0.005)
    parser.add_argument('--mutation_decay', type=float, default=0.90)

    return parser.parse_args()

```

adaptation_wtrack.py

```

def run_adaptation(label, checkpoint_path, env, checkpoints, walls, args):
    """
    Charge un checkpoint, lance N générations sur W-track,
    sauvegarde adaptation_w_track.pt dans le même dossier que le checkpoint.
    Retourne le chemin du fichier produit.
    """

    print(f"\n{'='*70}")
    print(f" ADAPTATION : {label}")
    print(f" Checkpoint : {checkpoint_path}")
    print(f"{'='*70}")

    # -- Population -----
    pop_manager = PopulationManager(
        n_population = args.population,
        n_rays       = args.n_rays,
        initial_mutation_rate = args.mutation_start,
        final_mutation_rate = args.mutation_end,
        mutation_decay  = args.mutation_decay,
        device          = args.device,
    )
    pop_manager.load_population_from_file(checkpoint_path)
    pop_manager.generation = 0

```

```

# -- Fitness tracker -----
fitness_tracker = FitnessTracker( checkpoints = checkpoints,
    spawn_point = (env.spawn_x, env.spawn_y, env.spawn_angle),
    n_cars = args.population, track_width = env.track_width, walls = walls,
    device = args.device)

# -- Boucle d'adaptation -----
training_loop = TrainingLoop( env = env, population_manager = pop_manager,
    fitness_tracker = fitness_tracker, frequency_show = args.frequency_show,
    walls = walls, max_laps = args.nb_laps, max_steps = args.nb_steps,
    phase2_gen = None, phase2_fitness = None,)

for gen in range(args.n_adapt_gen):
    print(f"\n Génération {gen + 1}/{args.n_adapt_gen}")
    stats = training_loop.run_generation(generation=gen)
    if stats is None:
        break
    print(
        f" Best={stats['best_fitness']:.1f} | "
        f"Avg={stats['avg_fitness']:.1f} | "
        f"Mutation={stats['mutation_rate']:.2%}"
    )

# -- Sauvegarde dans le même dossier que le checkpoint source -----
save_path = Path(checkpoint_path).parent / "adaptation_w_track.pt"
pop_manager.save_population(str(save_path))
print(f"\n Sauvegardé : {save_path}")
return str(save_path)

```

```

def main():
    args = parse_args()

    if args.device == 'cuda' and not torch.cuda.is_available():
        print("CUDA non disponible, passage en CPU")
        args.device = 'cpu'

# -- W-track chargé une seule fois, partagé entre les 3 agents -----
print("\nChargement du circuit W-track...")
configs = load_circuits(
    names = ['w_track'],
    n_cars = args.population,
    n_rays = args.n_rays,
    device = args.device,
)
cfg = configs[0]
env = cfg["env"]
checkpoints = cfg["checkpoints"]
walls = cfg["walls"]
print(f" Checkpoints : {len(checkpoints)}")

# -- 3 adaptations séquentielles -----
agents = [
    ("NASCAR", args.nascar),
    ("Speed Ring GT", args.gt),
    ("Multi-circuit", args.multi),
]

```

```
result_paths = {}
for label, ckpt in agents:
    result_paths[label] = run_adaptation(
        label, ckpt, env, checkpoints, walls, args
    )

# -- Commande show_comparison -----
print(f"\n{'='*70}")
print(" ADAPTATION TERMINÉE — commande pour la comparaison :")
print(f"{'='*70}")
print(
    f"\npython -m simulations.show_comparison \\\n"
    f"--nascar {result_paths['NASCAR']} \\\n"
    f"--gt {result_paths['Speed Ring GT']} \\\n"
    f"--multi {result_paths['Multi-circuit']}"
)

if __name__ == "__main__":
    main()
```